

Clase 3 - Visión Artificial

Visión por computador basada en machine learning

Agenda

- 1700 – 1900

- Repaso proyecto y clases anteriores
- Profundización del concepto de filtros para llegar a CNN (ML)
- Taller

- 1930 – 2130

- Ayuda en Tarea 1 – Procesamiento de Imágenes (Ayudante)
- Teoría de CV usando machine learning
- Taller

Situación de Proyectos – Uso de Roboflow

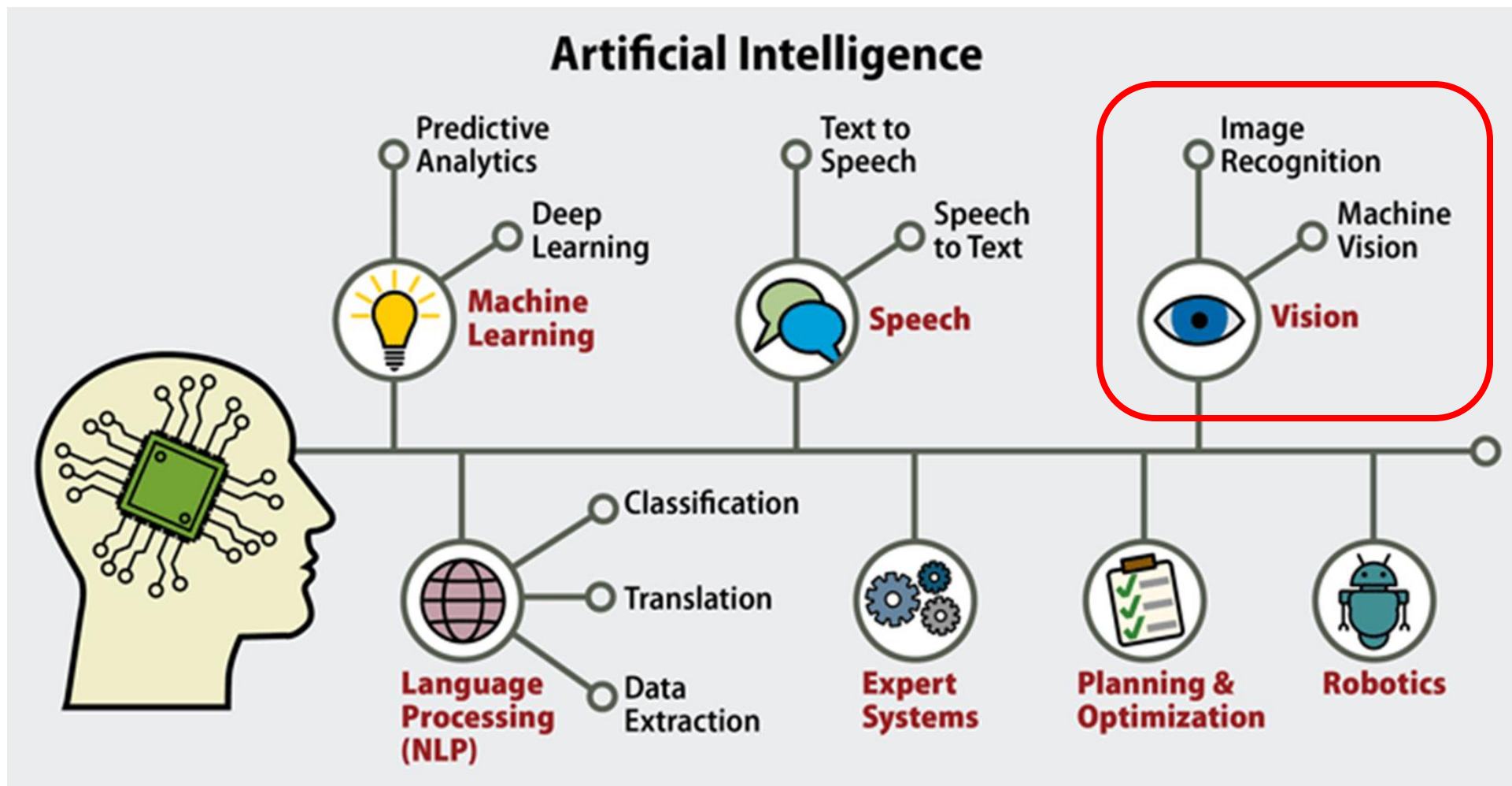
- Todos los alumnos deben subir el PDF de su proyecto
- Revisar proyectos no expuestos en clases
- se propone a los alumnos utilizar roboflow
 - <https://app.roboflow.com/gaia-s3pxc>
- Se presenta el próximo lunes, por 10 min, para la nota de Taller

Revisión de la Clase Anterior

Preguntas, tendencias, ayudar a los equipos

Para mejorar la experiencia de este curso haremos más talleres , uno de procesamiento de imágenes y otro de *deep learning* en grupo, con ayudante !

Este Campo esta dividido en habilidades para imitar la inteligencia humana:



¿Qué es la Visión por Computadora?

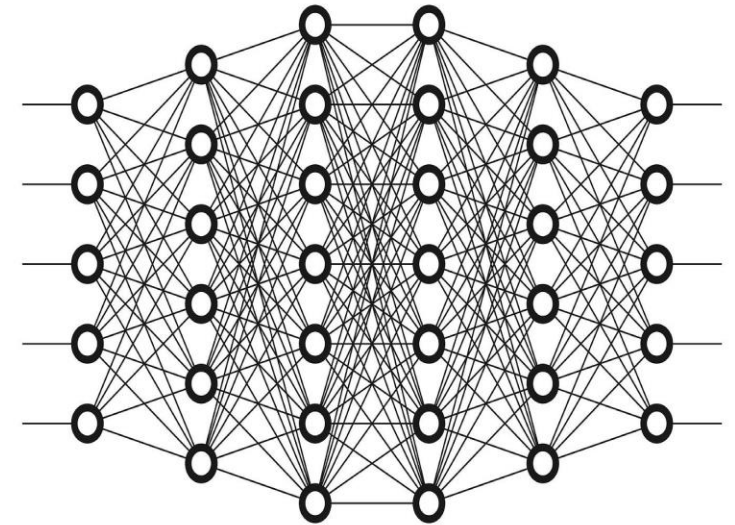
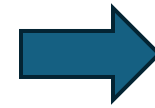
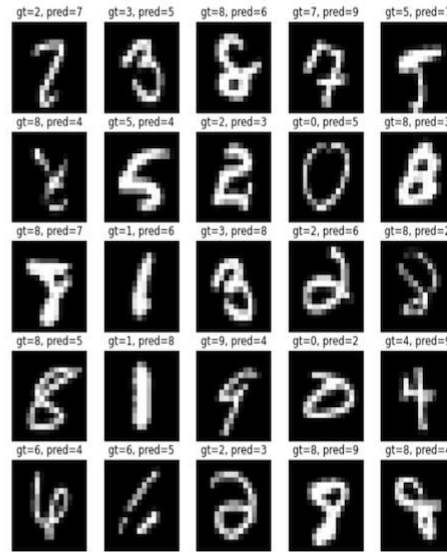
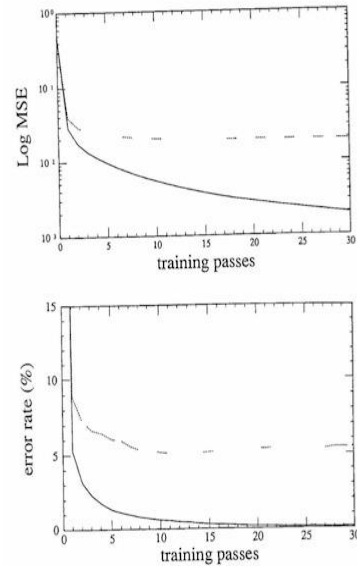
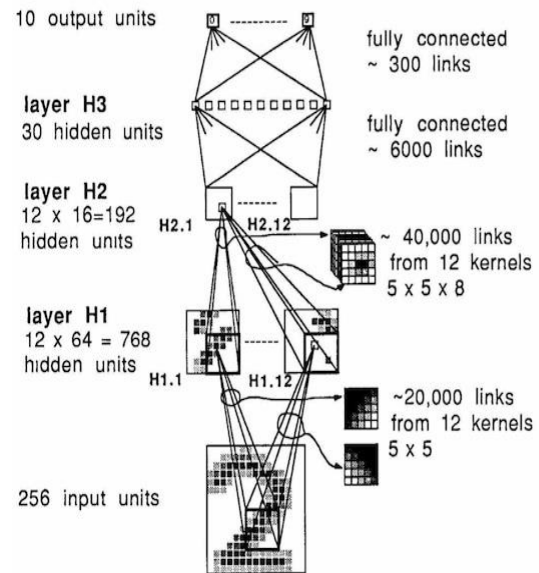
La **Visión por Computadora** es el campo que **permite a las máquinas obtener información significativa a partir de entradas visuales** como imágenes y videos.

Intenta replicar y extender las capacidades de percepción visual humana utilizando métodos computacionales.



La visión por computadora cierra la brecha entre los datos de píxeles brutos y la comprensión de alto nivel, de alto nivel, imitando y extendiendo las capacidades visuales humanas.

Qué es Deep Learning?



Perspectiva Histórica

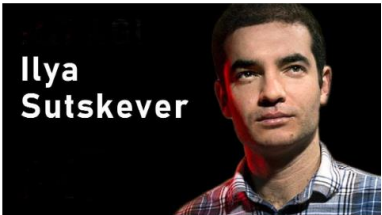
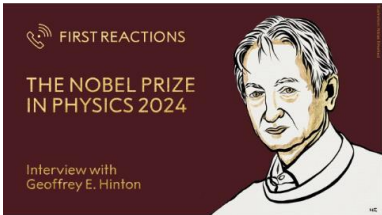


La visión por computadora ha evolucionado desde la simple detección de bordes hasta sofisticados sistemas de IA capaces de comprender escenas visuales complejas.

Cuándo cambi3 todo?



puter vision was in



Till now (Feb 15th, 2025) this paper has been cited more that 170,000 times!

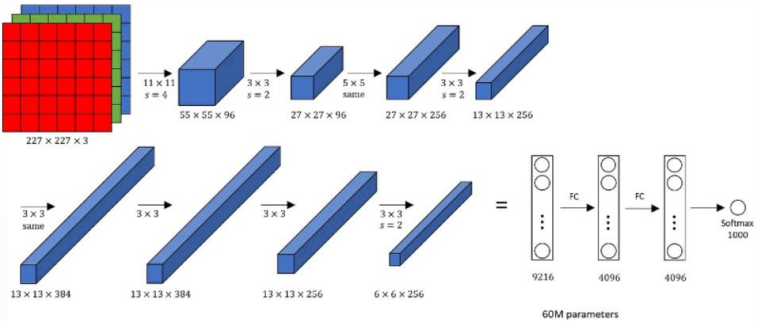


Geoffrey Hinton

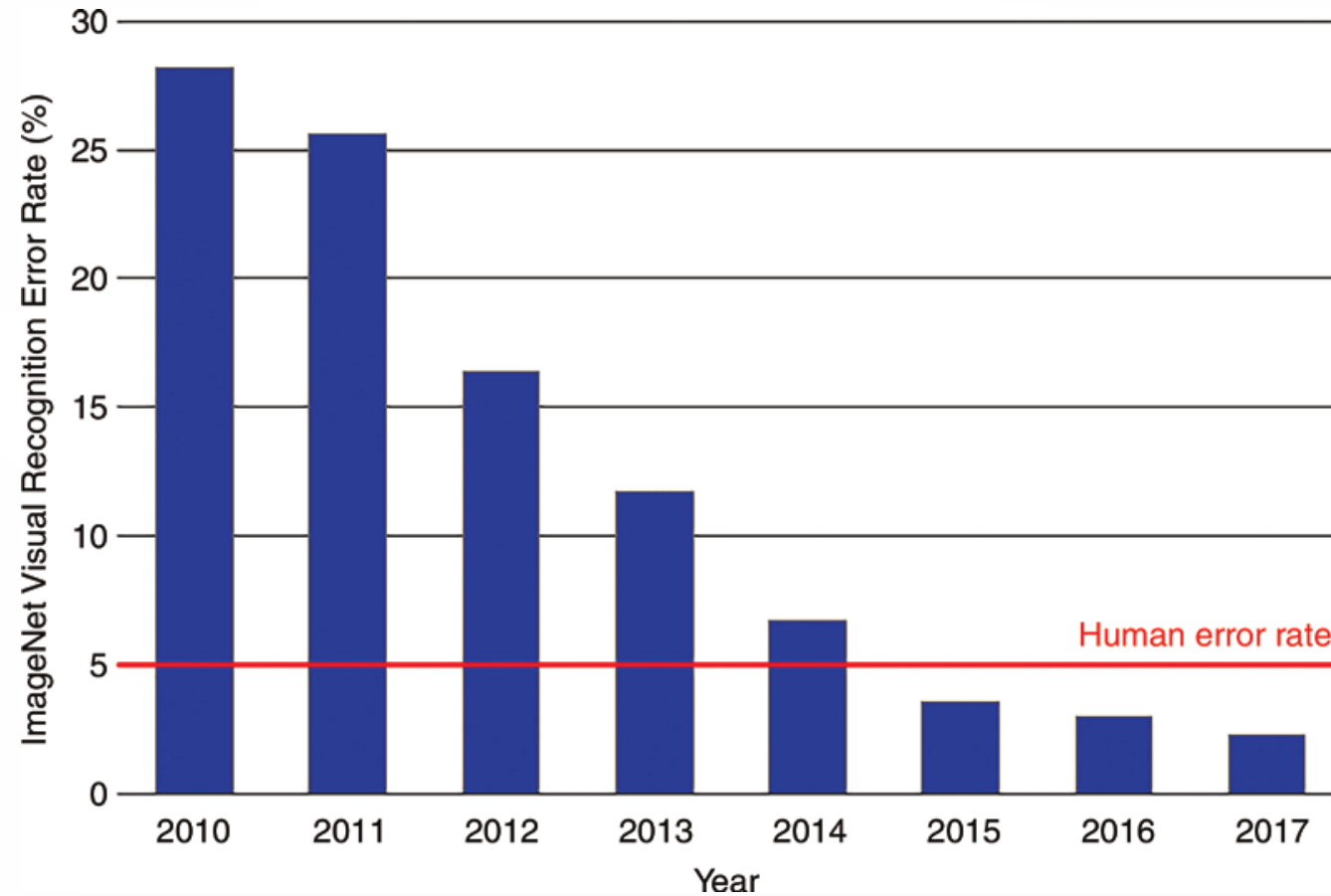
FOLLOW

Emeritus Prof. Computer Science, [University of Toronto](#)
Verified email at cs.toronto.edu - [Homepage](#)
[machine learning](#) [psychology](#) [artificial intelligence](#) [cognitive science](#) [computer science](#)

TITLE	CITED BY	YEAR
Imagenet classification with deep convolutional neural networks A Krizhevsky, I Sutskever, GE Hinton Advances in neural information processing systems 25	172486 [★]	2012



Cuándo cambió todo?

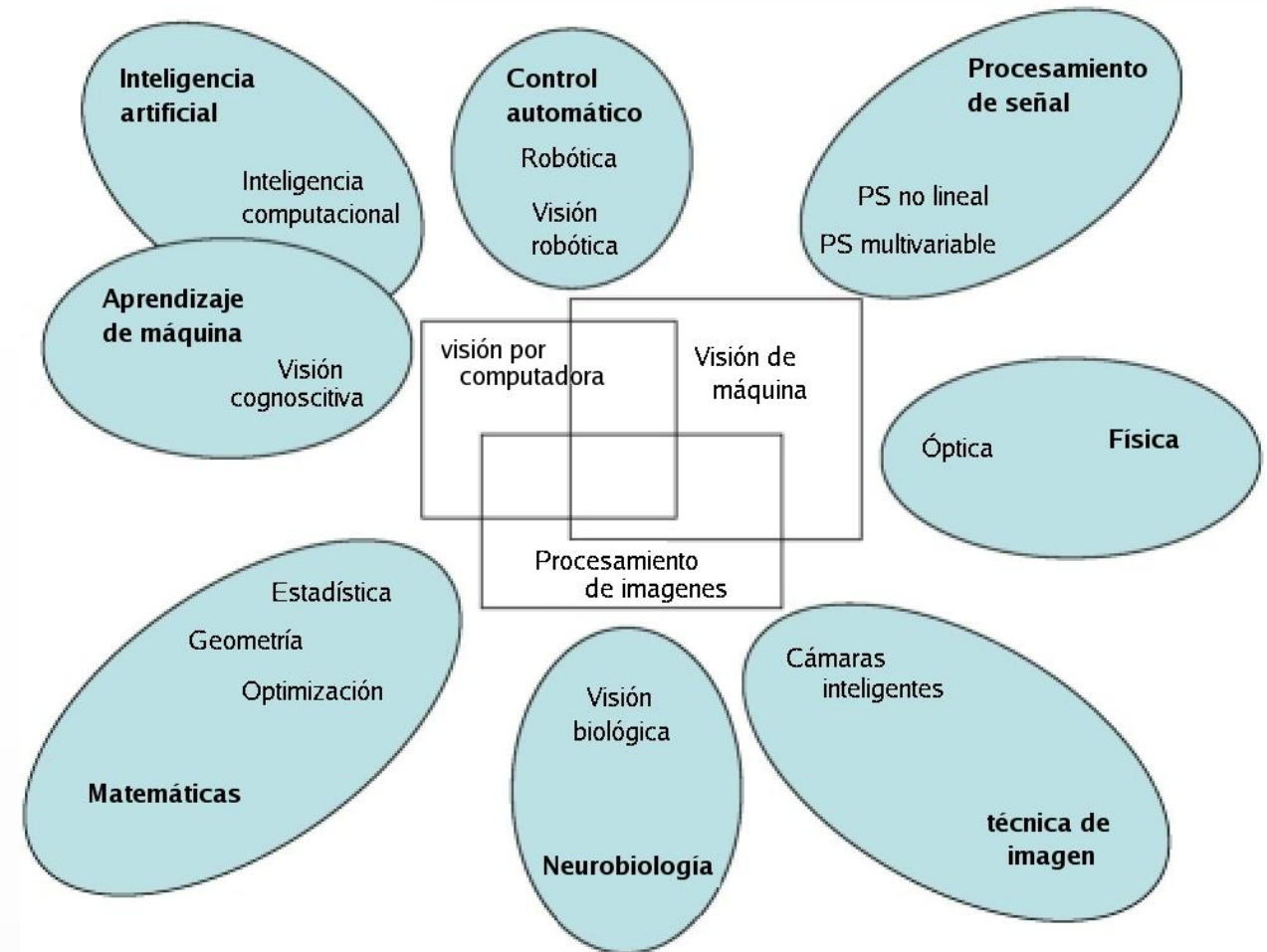


Tecnologías dentro de Computer Vision

Tecnologías dentro de Computer Vision

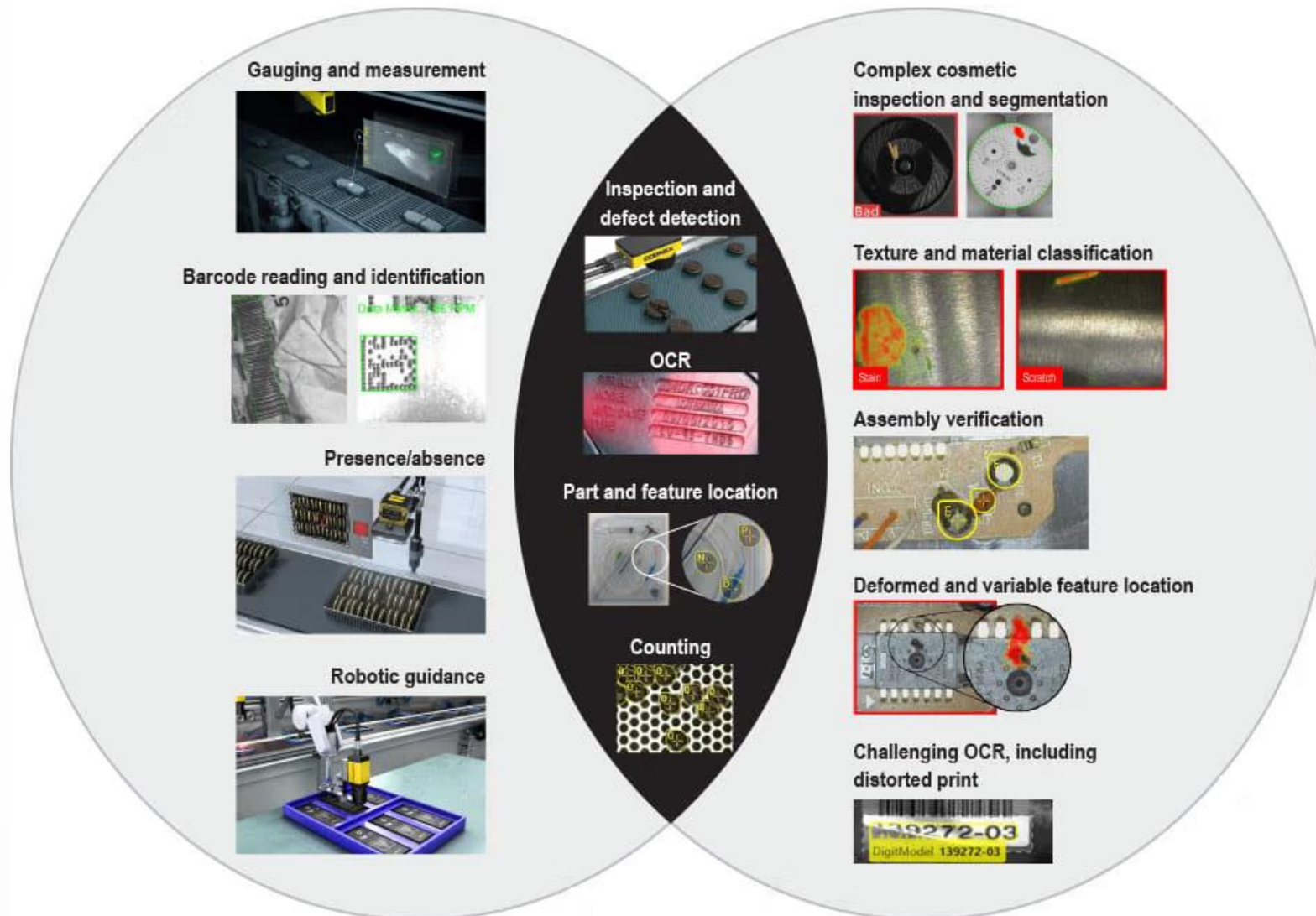
Este campo interdisciplinario combina conceptos de:

- Procesamiento y análisis de imágenes
- Aprendizaje automático y aprendizaje profundo
- Reconocimiento de patrones
- Robótica y sistemas de control

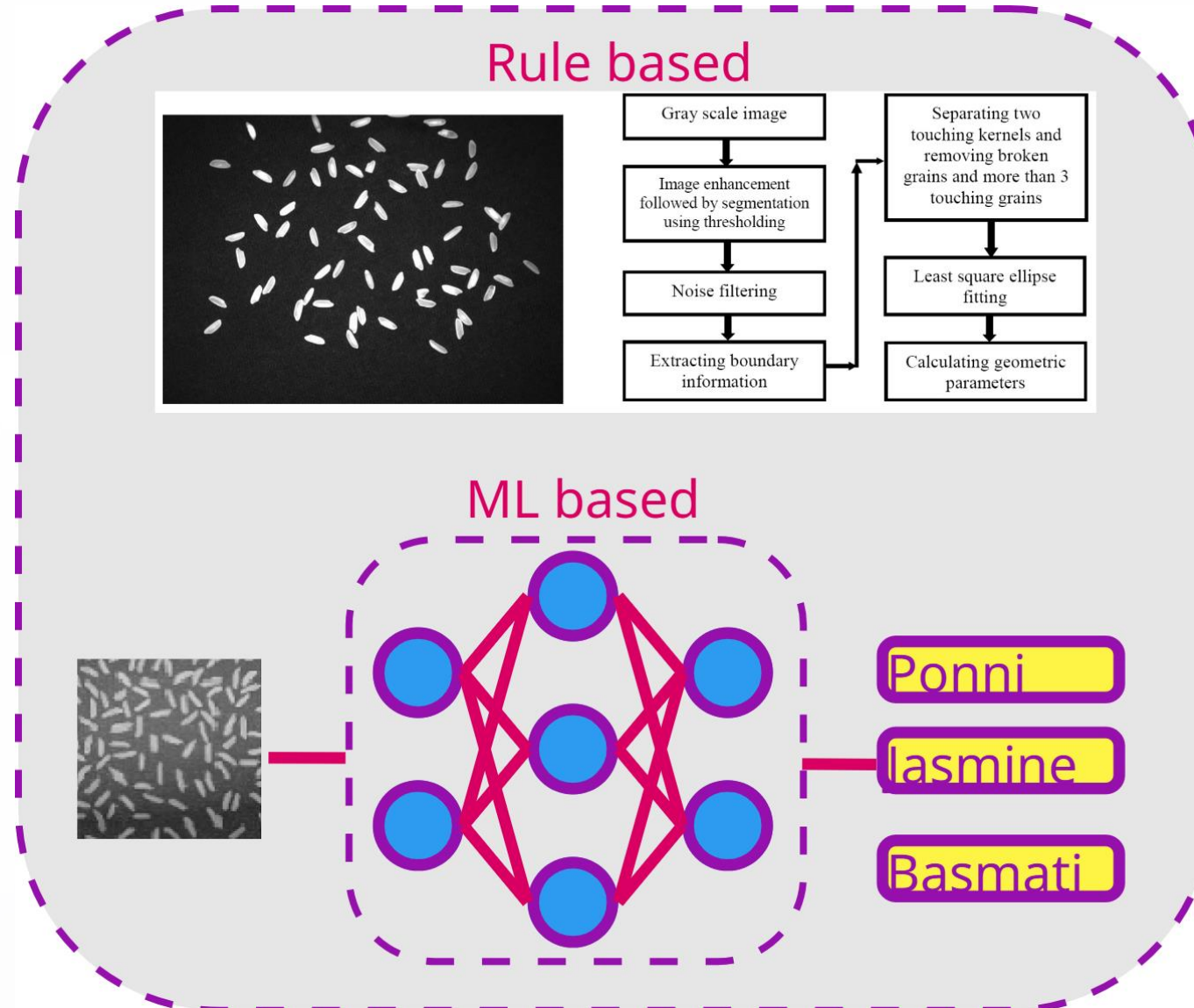


Visión por Computadora vs Vision Artificial

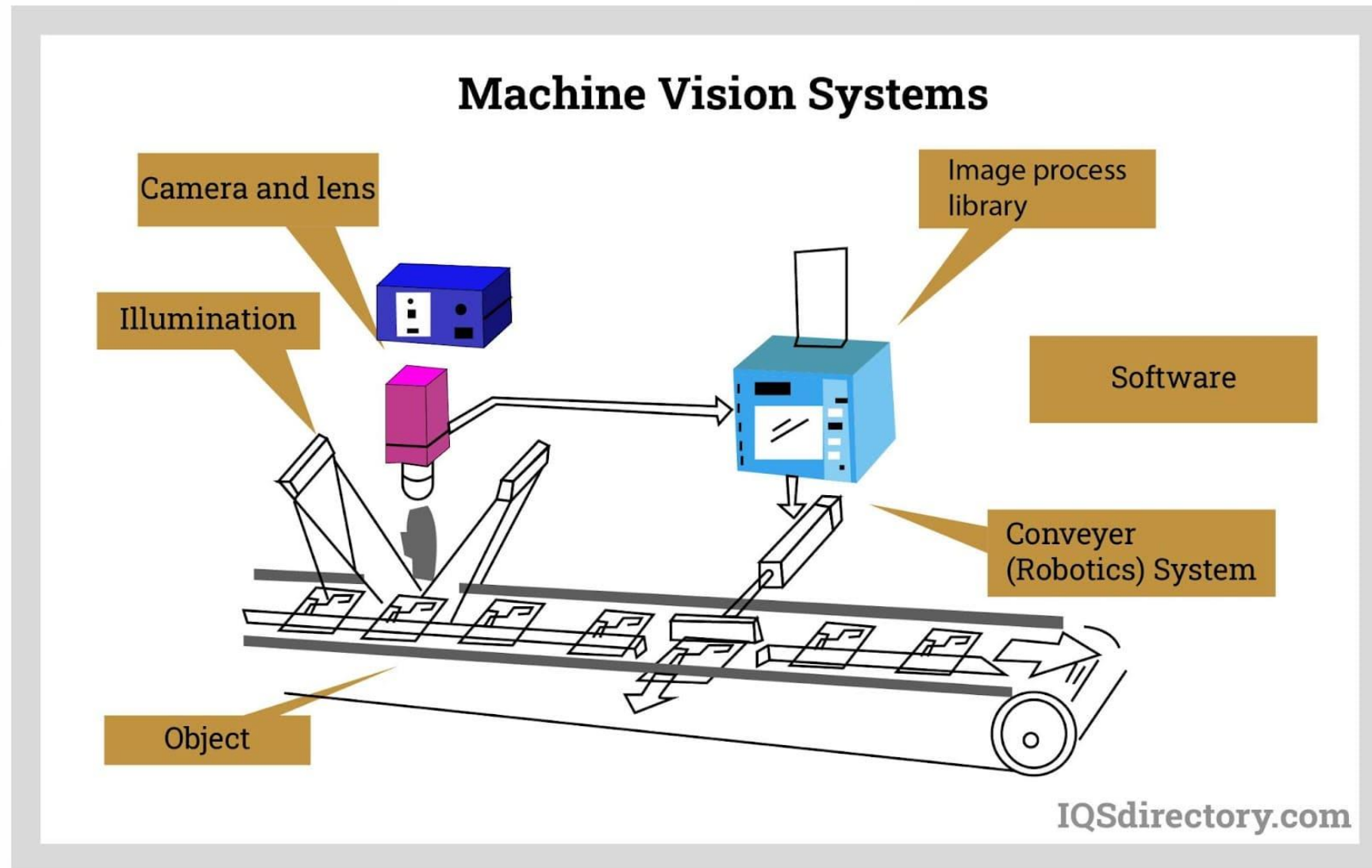
When to Deploy Traditional Machine Vision vs. Deep Learning-Based Image Analysis



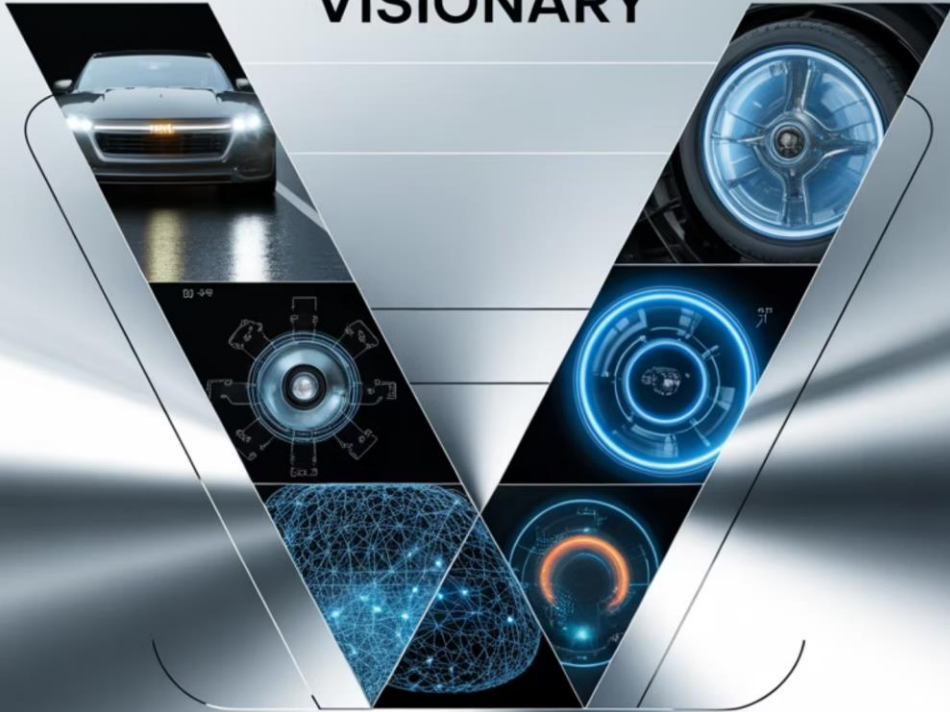
Visión por Computadora vs Vision Artificial



Visión por Computadora vs Machine Vision



VISIONARY



Aplicaciones en el Mundo Real

Inspección Industrial

Sistemas automatizados de control de calidad que detectan defectos de fabricación con mayor precisión que los inspectores humanos. Estos sistemas operan 24/7 sin fatiga, manteniendo estándares de calidad consistentes.

Vehículos Autónomos

Sistemas de visión que detectan peatones, señales de tráfico, marcas de carril y otros vehículos en tiempo real, permitiendo una navegación segura en entornos complejos.

Diagnóstico Médico

Análisis impulsado por IA de imágenes médicas para detectar tumores, fracturas y otras anomalías, asistiendo a los médicos en el diagnóstico temprano y la planificación del tratamiento.

Seguridad y Vigilancia

Sistemas de reconocimiento facial, seguimiento de personas y detección de anomalías que mejoran la seguridad en espacios públicos e instalaciones privadas.

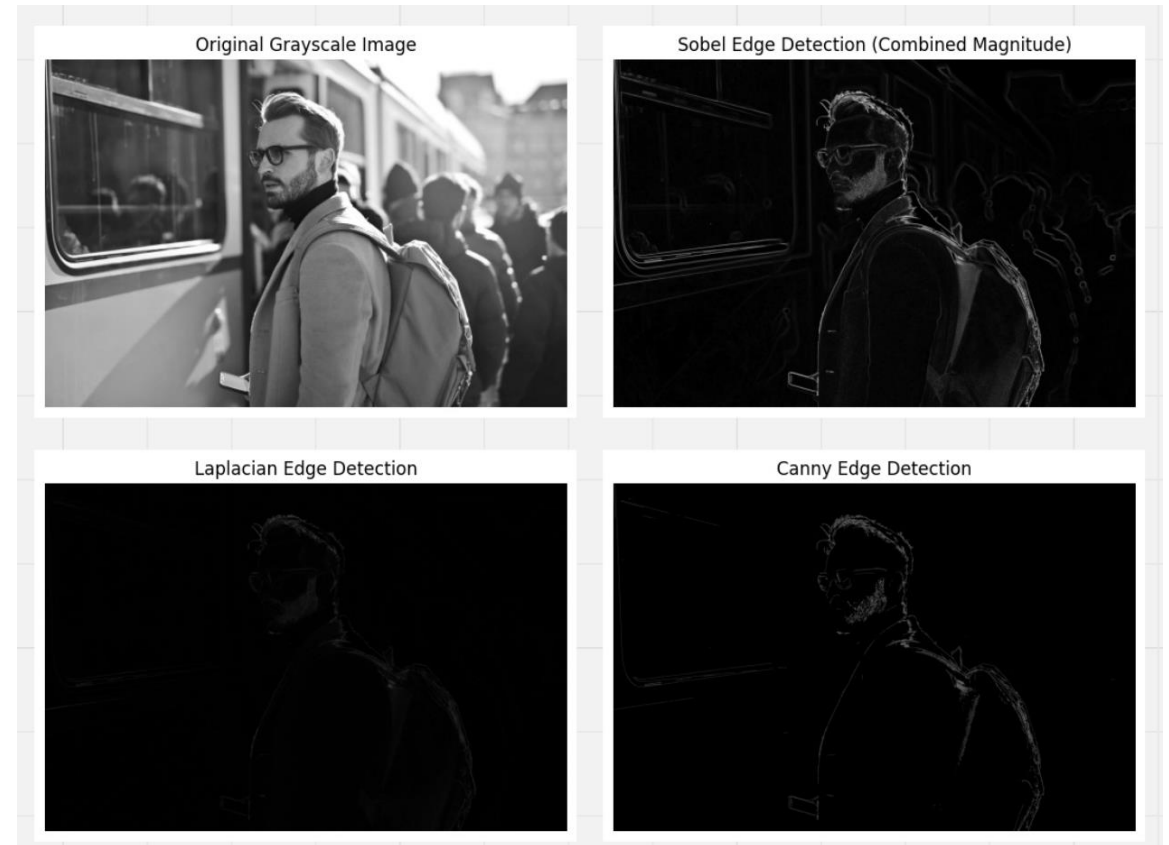


A close-up photograph of a person's hand holding a camera lens. The lens is dark and shows concentric circular patterns. The background is blurred with warm, orange-toned light. The text 'Procesamiento de Imágenes' is overlaid in the center in a white, sans-serif font.

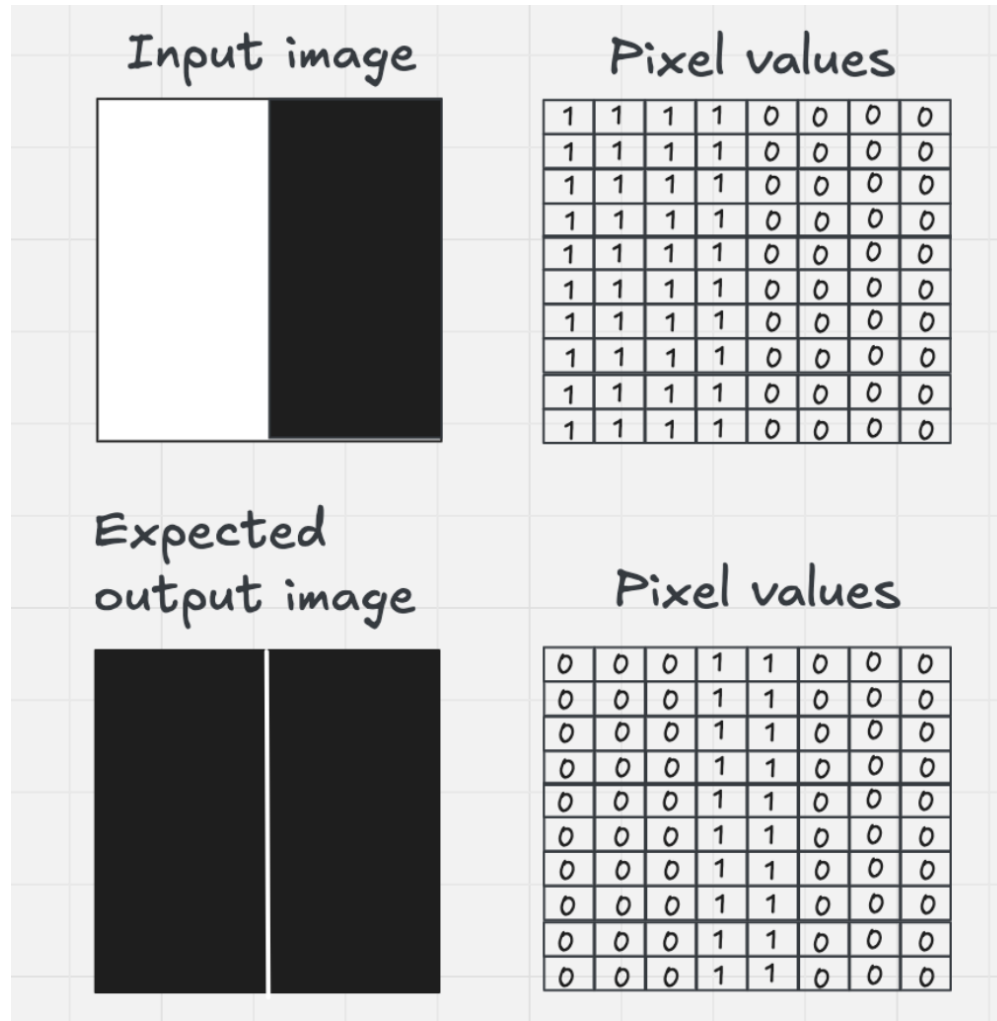
Procesamiento de Imágenes

... antes del 2010

- Antes de 2010, las tareas de visión por computadora se realizaban utilizando filtros de imagen. Los filtros le permiten identificar características específicas dentro de una imagen.
- Algunos filtros podrían estar destinados a la detección de bordes, otros podrían estar destinados a detectar formas circulares.



En algunos casos, queríamos detectar un borde en la imagen



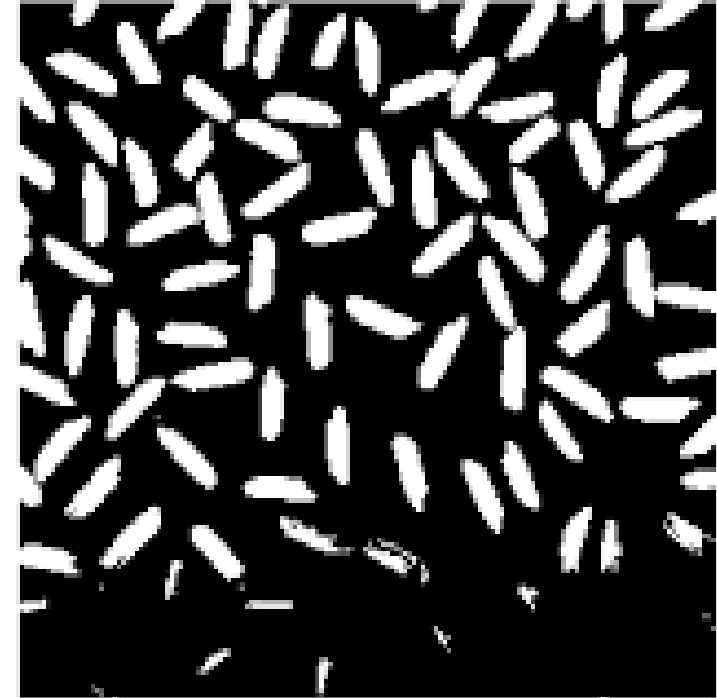
Segmentación Tradicional

La segmentación divide una imagen en regiones significativas, facilitando el análisis por separado de objetos o áreas de interés.

Métodos Clásicos:

- **Umbralización global (Otsu):** Determina automáticamente un umbral óptimo basado en la distribución bimodal del histograma
- **Umbralización adaptativa:** Aplica umbrales locales que se adaptan a diferentes regiones de la imagen
- **Clustering (K-means):** Agrupa píxeles según su similitud en el espacio de características

La segmentación es fundamental para aislar objetos antes de su análisis detallado, como en esta en esta separación objeto-fondo.

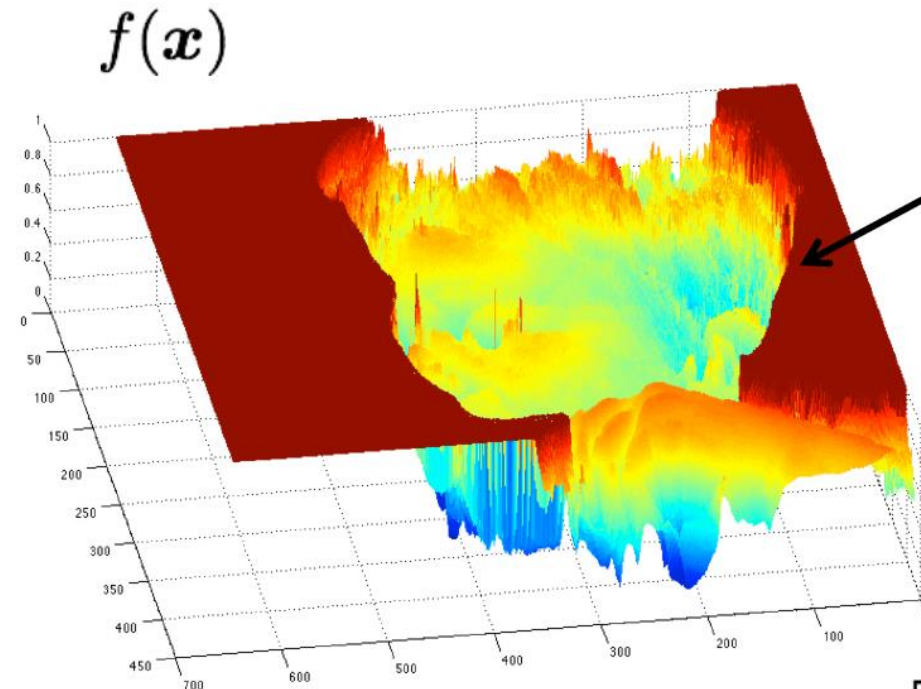


Detección de Bordes: Localizando Transiciones

Los bordes representan cambios significativos en la intensidad de la imagen y suelen corresponder a límites entre objetos o características importantes.



grayscale image



Very sharp
discontinuities
in intensity.

$$\text{domain } \mathbf{x} = \begin{bmatrix} x \\ y \end{bmatrix}$$

Estas técnicas proporcionan información crucial sobre la estructura geométrica para reconocimiento de formas, segmentación avanzada y análisis de contornos.

Detección de Bordes: Localizando Transiciones

Los bordes representan cambios significativos en la intensidad de la imagen y suelen corresponder a límites entre objetos o características importantes.

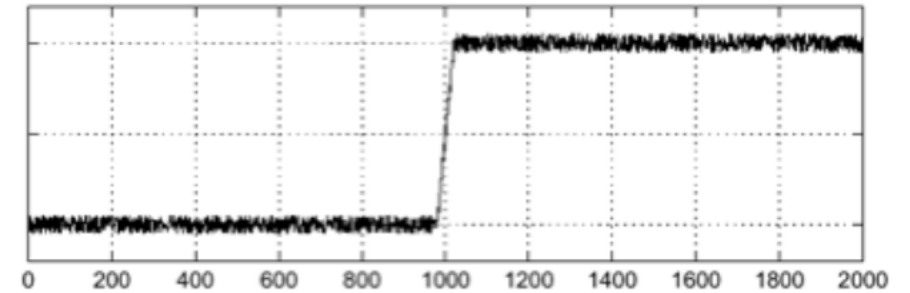
Filtro Sobel

Calcula los gradientes horizontal y vertical mediante la convolución con dos máscaras de 3×3 , estimando tanto la magnitud como la dirección del borde.

Algoritmo Canny

Proceso multi-etapa que incluye suavizado gaussiano, cálculo de gradientes, supresión de no-máximos y umbralización con histéresis.

intensity plot



Laplaciano

Detecta bordes buscando cruces por cero en la segunda derivada de la imagen, identificando cambios rápidos en el gradiente.

Ejemplos del Filtro Sobel

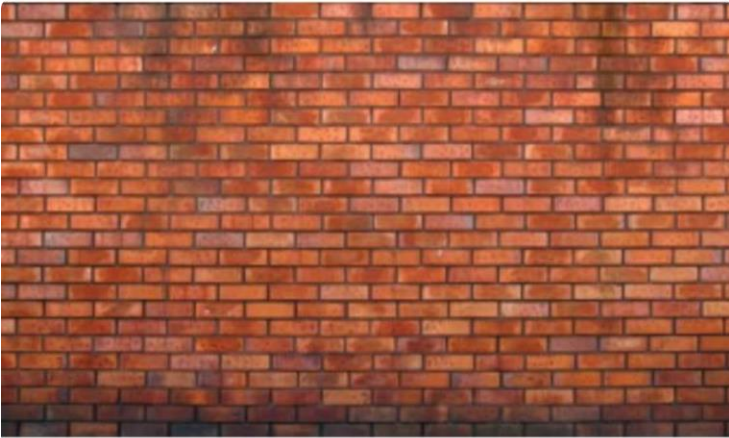


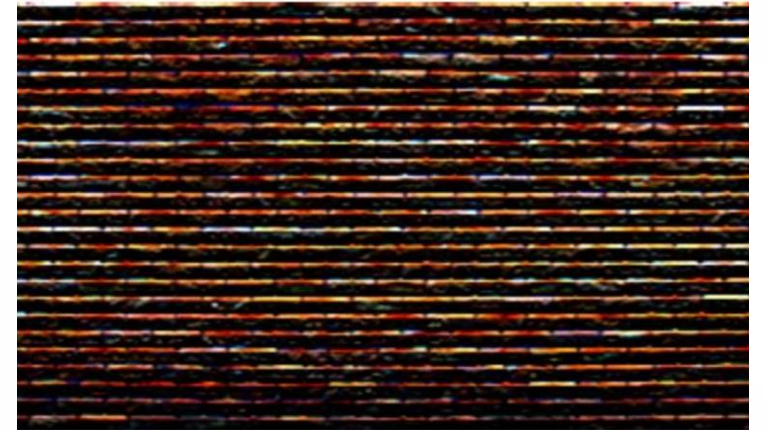
Imagen Original

Antes de la detección de bordes



Sobel Horizontal

Detecta bordes verticales



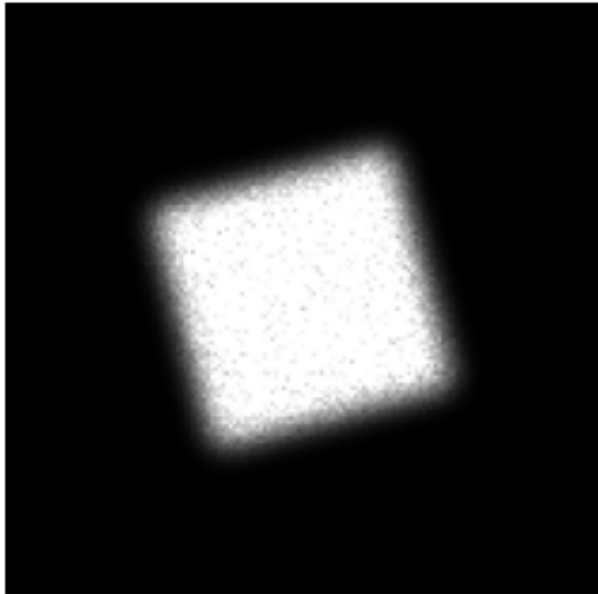
Sobel Vertical

Detecta bordes horizontales

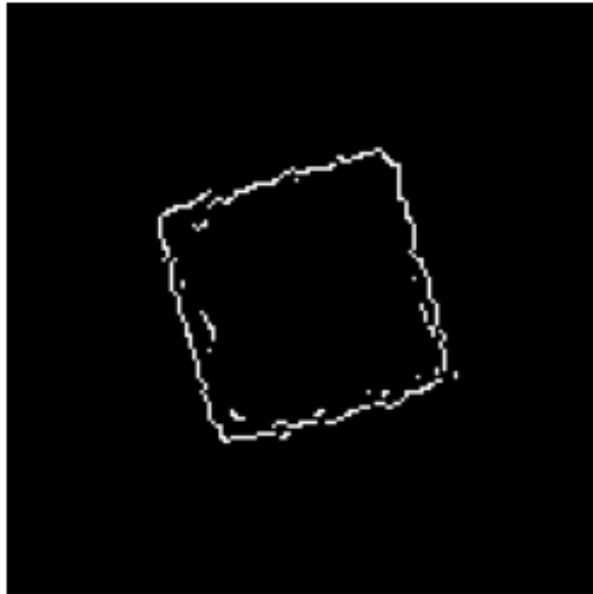
La magnitud del gradiente completa se puede calcular tomando la raíz cuadrada de la suma de los cuadrados de los gradientes horizontal y vertical.

Ejemplos del Filtro Canny

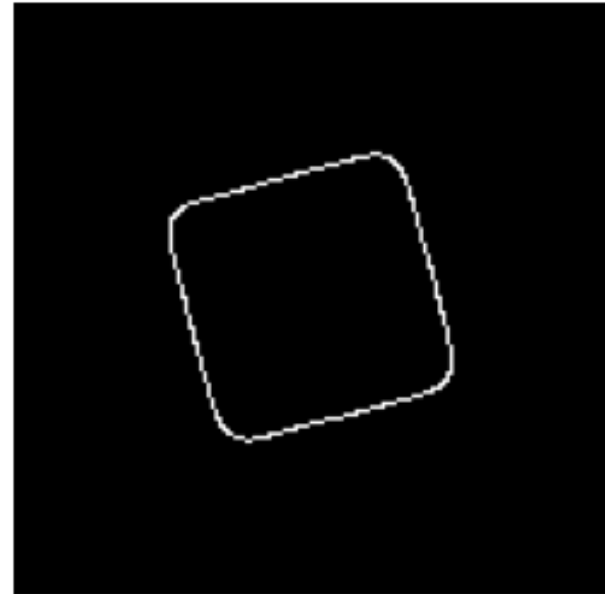
noisy image



Canny filter, $\sigma = 1$



Canny filter, $\sigma = 3$



Sistemas de Expertos tradicionales

- En los sistemas clásicos de CV basados en expertos, tratamos de definir la lógica (*como los filtros*) para decidir qué es una flor o qué es un ser humano a partir de una imagen. Aunque era bueno para casos de uso específicos, era muy difícil hacer que los sistemas clásicos basados en expertos funcionaran para una multitud de casos extremos.

Desde Filtros (kernel) a Redes Neuronales Convolucionales (CNN)

Filtrado Lineal Invariante al Desplazamiento

Combinación Lineal

Cada píxel de salida es una suma ponderada de los píxeles de entrada en su vecindad

Basado en Kernel

Los pesos son determinados por el kernel del kernel del filtro (también llamado máscara o máscara o ventana)

Invariante al Desplazamiento

El mismo kernel se aplica en cada ubicación de ubicación de píxel en toda la imagen

Este enfoque forma la base de muchas operaciones de procesamiento de imágenes, incluyendo desenfoque, nitidez y detección de bordes.

$$\text{kernel } g[\cdot, \cdot] = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Propiedades

- Reemplaza cada píxel con el promedio de su vecindario de 3x3
- Todos los pesos son iguales (uniformes)
- Crea un efecto de desenfoque/suavizado
- Reduce el ruido pero también reduce el detalle

Visualización del Proceso de Filtrado

$$\frac{1}{9} \begin{matrix} & g[\cdot, \cdot] \\ \text{kernel} & \begin{bmatrix} | & | & | \\ | & | & | \\ | & | & | \end{bmatrix} \end{matrix}$$

image			$f[\cdot, \cdot]$						
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

output			$h[\cdot, \cdot]$						
	0								

El proceso de filtrado implica:

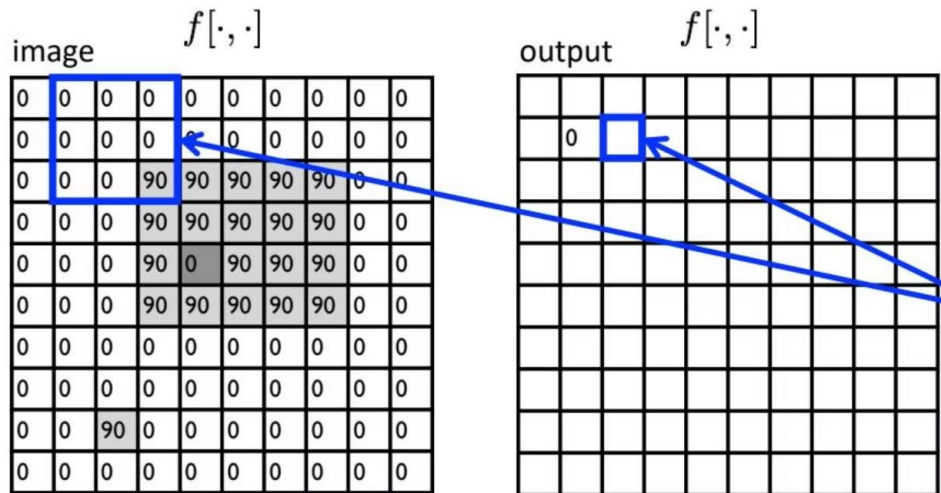
1. Centrar el kernel en cada posición de píxel.
2. Multiplicar cada valor del kernel por el píxel de imagen correspondiente.
3. Sumar estos productos para producir el valor del píxel de salida.
4. Moverse al siguiente píxel y repetir.

Esta operación se expresa matemáticamente como: $h[m,n] = \sum_{k,l} g[k,l] f[m+k,n+l]$

Filtrado Paso a Paso

Multiplicar Valores

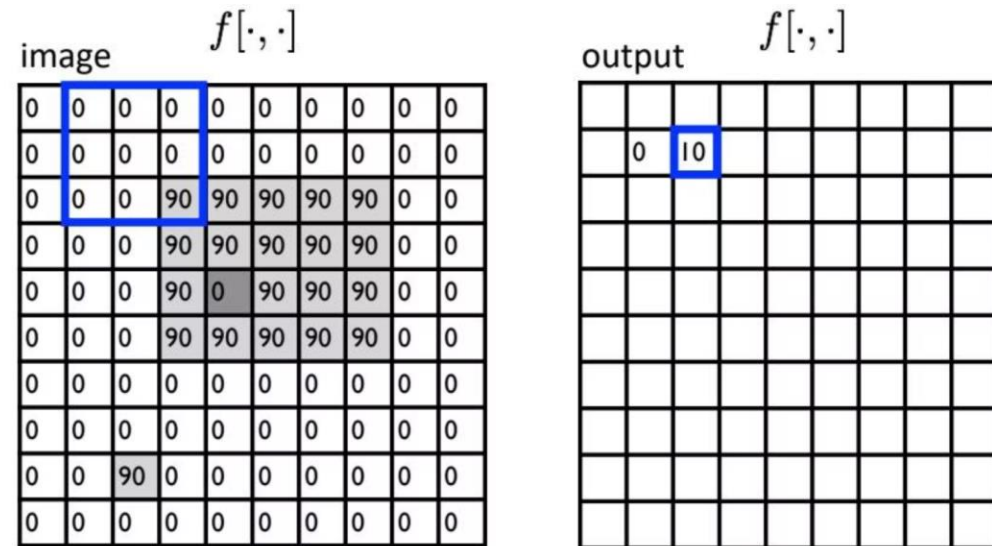
Multiplica cada valor del núcleo con el valor del píxel correspondiente de la imagen



shift-invariant
as the pixel
shifts, so does
the kernel

Repetir Proceso

Pasa a la siguiente posición del píxel y repite el proceso



Otras aplicaciones de procesamiento espacial

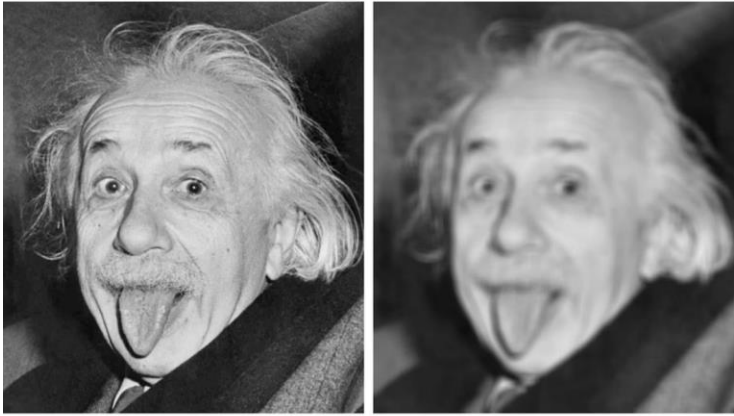


Imagen Original

Antes de filtrar



Filtro de Caja Pequeño

Kernel 3×3 - desenfoque suave



Filtro de Caja Grande

Kernel 7×7 - desenfoque más fuerte

Los kernels más grandes producen efectos de desenfoque más pronunciados, ya que promedian los píxeles en un área más amplia.

Por qué usamos Filtros ?

- Los filtros pueden realizar varias operaciones útiles en el procesamiento de imágenes.
- **Extracción de características:** para detectar bordes, esquinas o texturas
- **Reducción de ruido:** Suavizado de filtros como el desenfoque gaussiano
- **Mejora:** Algunos filtros agudizan o mejoran los bordes para realizar más análisis
- **Reconocimiento de patrones:** Mucho antes de las redes neuronales, muchas tareas de CV implicaban filtros cuidadosamente elegidos más heurísticas basadas en reglas.

Eliminación de ruido

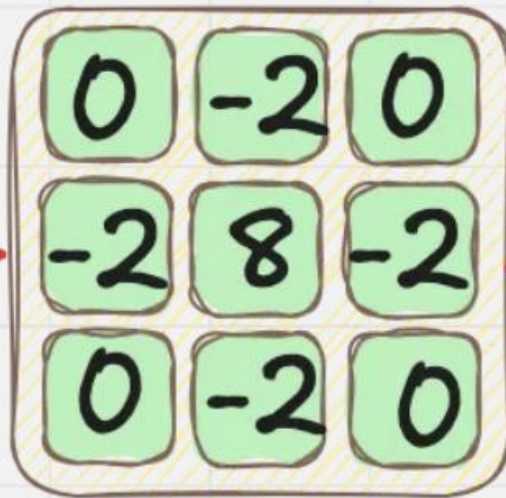
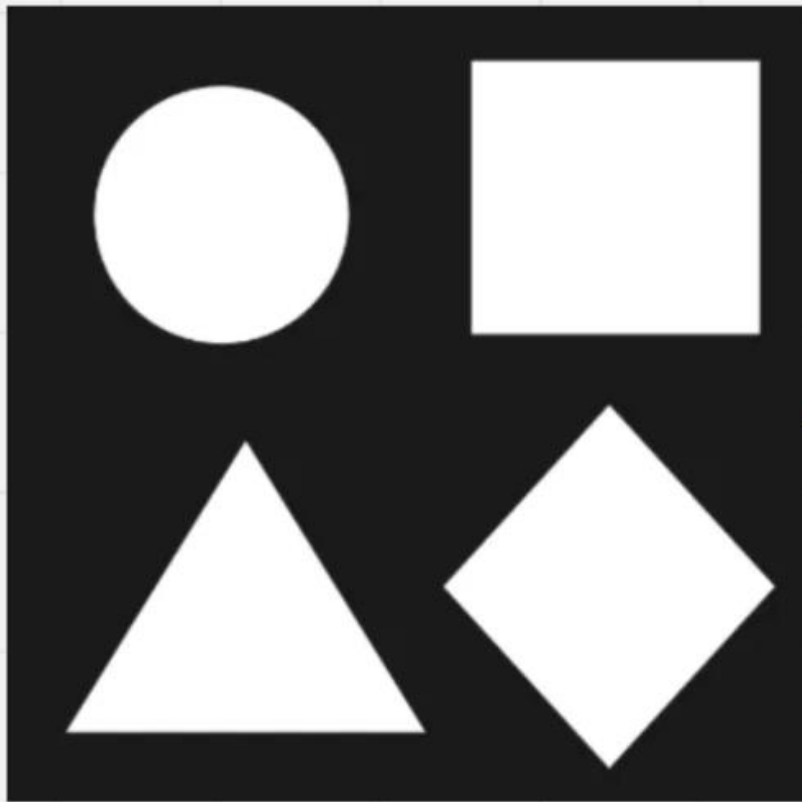


Denoising

0.0625	0.125	0.0625
0.125	0.25	0.125
0.0625	0.125	0.0625

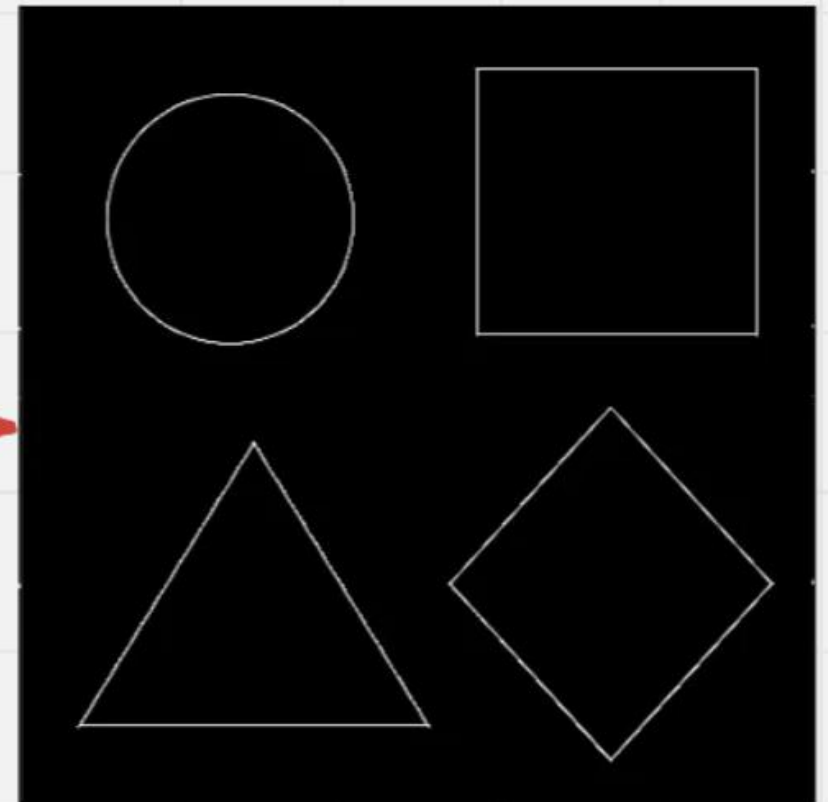


Detección de bordes



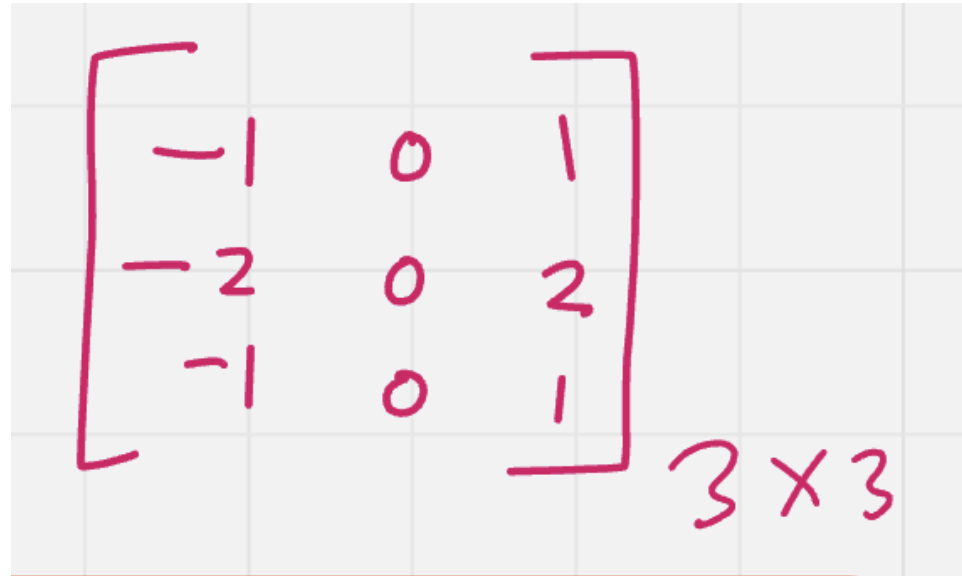
A 3x3 kernel matrix used for edge detection, with values in green boxes:

0	-2	0
-2	8	-2
0	-2	0



Construir filtros

- Cada vez que vemos un filtro como este:



A handwritten 3x3 matrix is shown on a light gray grid background. The matrix is enclosed in large square brackets. The elements are: the first row contains -1, 0, and 1; the second row contains -2, 0, and 2; and the third row contains -1, 0, and 1. To the right of the matrix, the text "3x3" is written in red, indicating the dimensions of the matrix.

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} 3 \times 3$$

- Es natural pensar, de donde salieron? Porque son esos números??

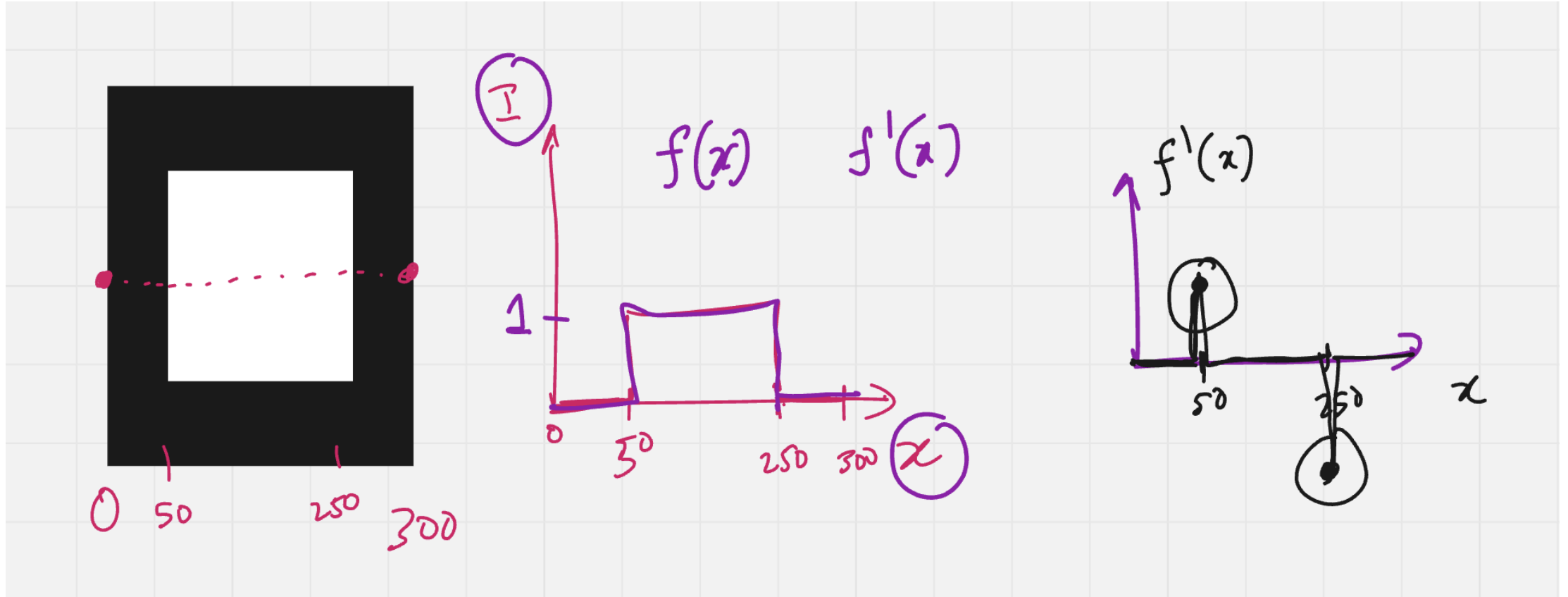
Filtros Clásicos, visto en clase anterior (Sobel, Laplacian, Gaussian, etc) no salieron del aire... obviamene

- Proviene de una mezcla de **fundamentos matemáticos** (diferencias finitas, derivadas, suavizado) y **experimentos prácticos** con imágenes reales.
- Vayamos bajo el capó y veamos cómo surgieron estas formas y valores.

Filtros Clásicos, visto en clase anterior (Sobel, Laplacian, Gaussian, etc) no salieron del aire... obviamente

- Un **borde** es básicamente un lugar donde la intensidad de una imagen cambia abruptamente.
- Matemáticamente, eso es una **derivada**, una medida de qué tan rápido cambia una función **f (x)**.
- Para una imagen cuyos valores de píxel se pueden describir como **I(x,y)**, tiene derivadas parciales $\partial I / \partial x$ (cambio en la dirección x) y $\partial I / \partial y$ (cambio en la dirección y).
- En una imagen digital, no tenemos funciones continuas sino píxeles discretos. Por lo tanto, aproximamos las derivadas usando **diferencias finitas**.

Pizarra



Diferencia finita 1D simple

Si desea aproximar la derivada d/dx de una función f en un punto x , puede hacer:

$$f'(x) \approx \frac{f(x+1) - f(x-1)}{2}$$

- ¿Por qué? Porque si imagina la función como una secuencia de valores, la diferencia entre vecinos puede representar qué tan rápido cambia la función.

Traducción de diferencias finitas a 2D

- Pero las imágenes reales tienen ruido y requieren cálculos más estables, por lo que modificamos estas simples diferencias en filtros más sofisticados (*como Sobel o Prewitt*)

Filtro Sobel

Sobel Edge Detection

Input Image:

10	9	9	4	0
0	6	6	2	2
5	9	8	4	3
7	5	5	4	3
8	10	8	5	0

Sobel Kernels

-1	0	1
-2	0	2
-1	0	1

X-Direction

1	2	1
0	0	0
-1	-2	-1

Y-Direction

Filtro Prewitt

Prewitt Edge Detection

Input Image:

10	9	9	4	0
0	6	6	2	2
5	9	8	4	3
7	5	5	4	3
8	10	8	5	0

Prewitt Kernels

-1	0	1
-1	0	1
-1	0	1

X-Direction

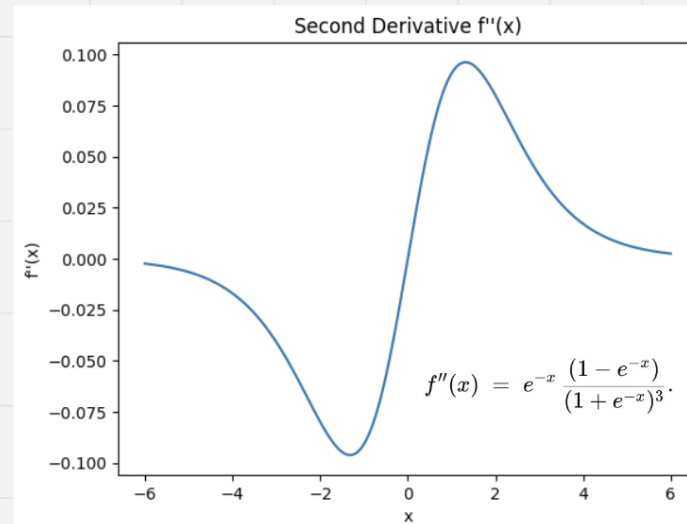
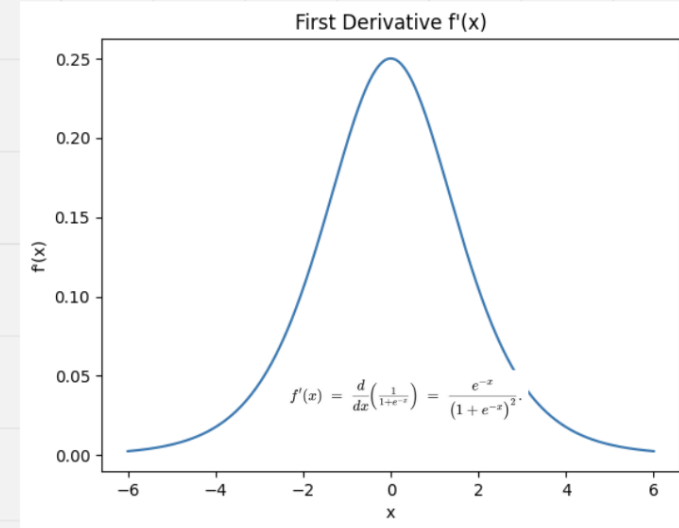
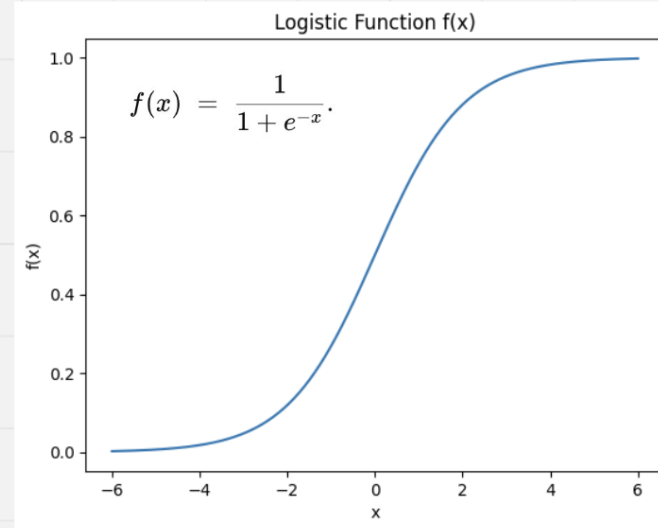
1	1	1
0	0	0
-1	-1	-1

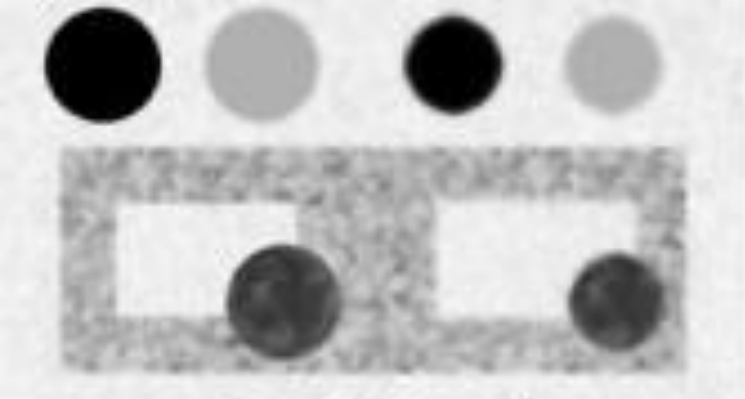
Y-Direction

Filtro laplaciano

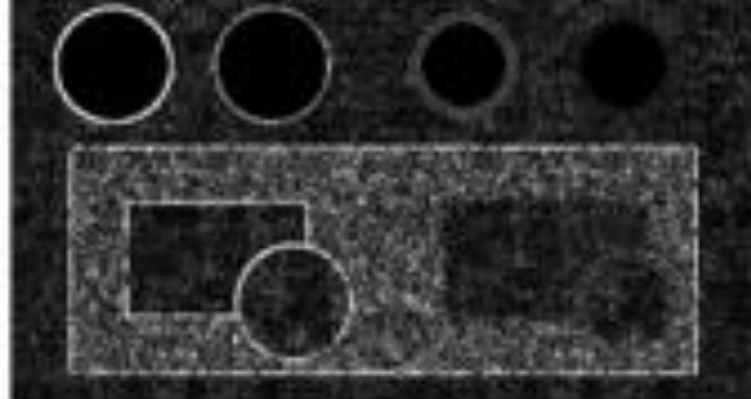
- Los bordes a menudo se vuelven **más claros** cuando miras la segunda derivada de una imagen.
- La segunda derivada amplifica los cambios rápidos de intensidad de manera más agresiva que la primera derivada.
- Básicamente, busca *qué* tan rápido está cambiando el gradiente en sí. Esto a menudo produce "picos" más nítidos en los bordes, haciéndolos parecer más distintos en un mapa de segunda derivada.

Laplaciano





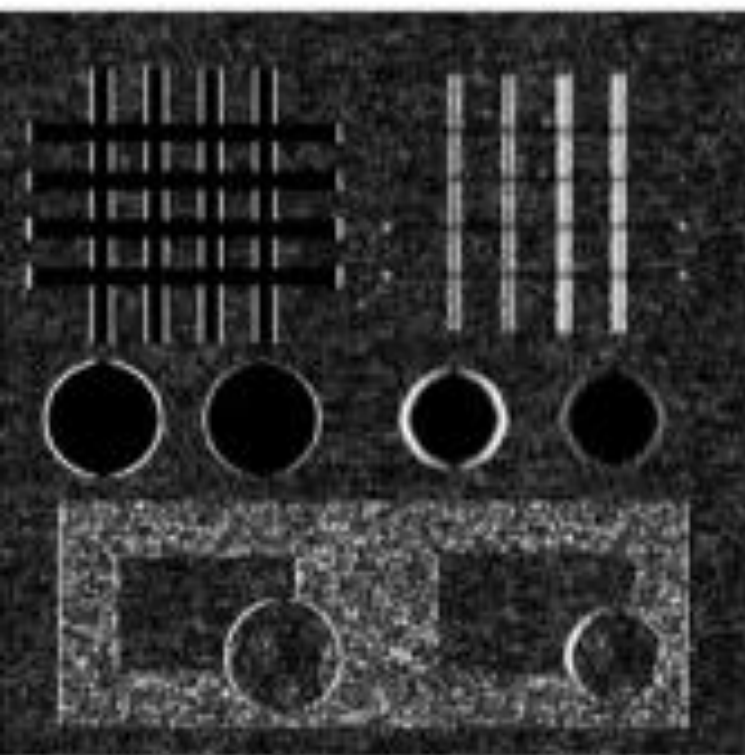
Original



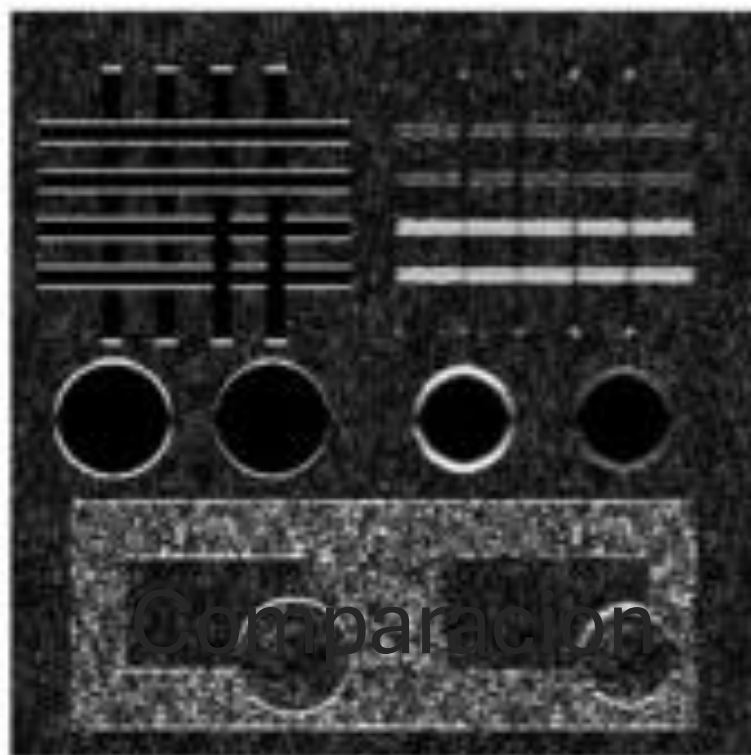
Laplacian



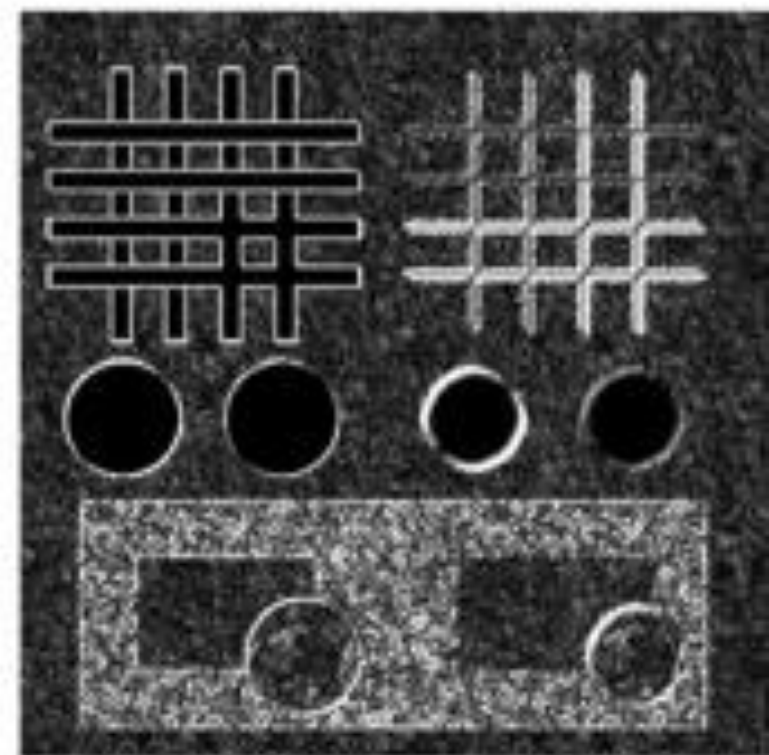
Canny



Sobel X



Sobel Y



Sobel X+Y

Taller de Procesamiento de Imágenes

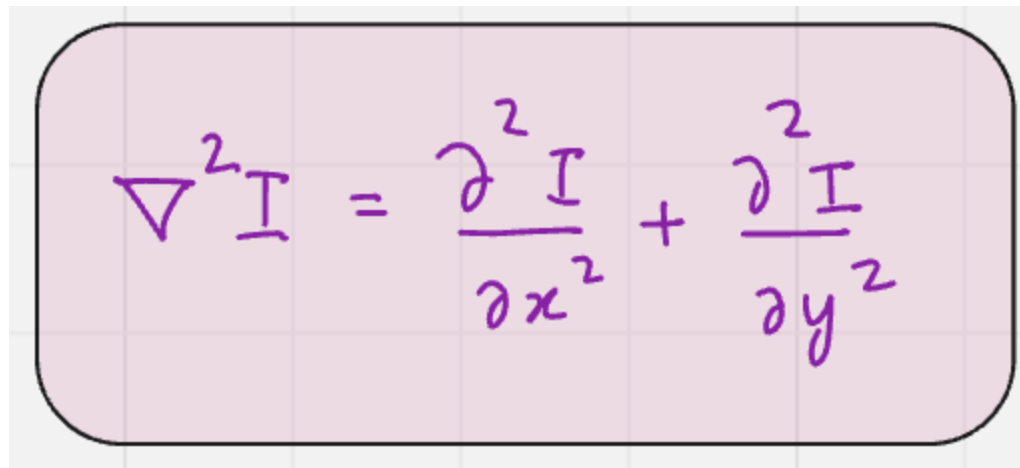
- Entender el Filtro Laplaciano

- El **kernel laplaciano** que se muestra arriba no es aleatorio. Es un resultado directo de tomar la segunda derivada en las direcciones x e y en una imagen discreta. Numéricamente, corresponde a sumar los cuatro vecinos directos (arriba, abajo, izquierda, derecha) y restar 4 veces el píxel central.

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

Entender el Filtro Laplaciano

- En el corazón del **operador laplaciano** se encuentra un principio básico: **destaca dónde la intensidad cambia rápidamente en todas las direcciones**. En matemáticas, el laplaciano es una operación de segunda derivada que, en forma continua, se ve así:


$$\nabla^2 I = \frac{\partial^2 I}{\partial x^2} + \frac{\partial^2 I}{\partial y^2}$$

Pizarra

$$\begin{aligned}\frac{\partial^2 f}{\partial x^2} &\approx \frac{f'(x+1) - f'(x)}{1} \\ &= \frac{\left[\frac{f(x+1) - f(x)}{1} \right] - \left[\frac{f(x) - f(x-1)}{1} \right]}{1} \\ &= f(x+1) - 2f(x) + f(x-1)\end{aligned}$$

① -2 +1

$$\frac{\partial^2 f}{\partial y^2} \approx f(y+1) - 2f(y) + f(y-1)$$

0 +1 0
1 -4 1
0 +1 0

→

0 +1 0
1 -2 1
0 +1 0

→

0 +1 0
1 -4 1
0 +1 0

→ dark to light (or vice versa)
→ creating strong

Filtro Laplaciano

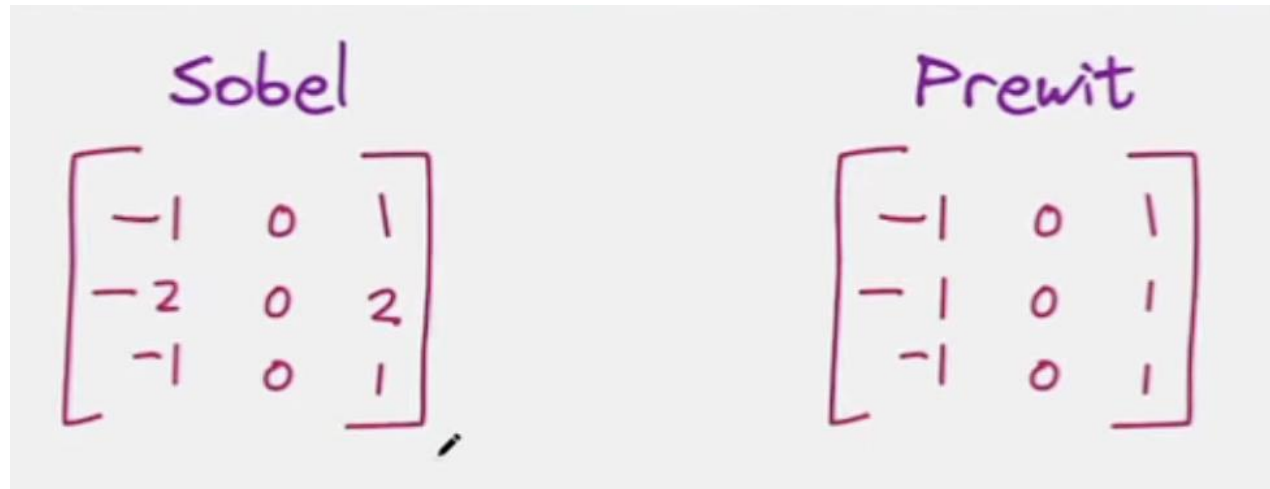
- Un **borde** en una imagen es donde los valores de brillo cambian rápidamente de oscuro a claro (o viceversa). El operador laplaciano enfatiza estas transiciones, a menudo creando respuestas fuertes exactamente donde la intensidad cambia más rápido.

$$\nabla^2 f = \frac{\partial^2 f}{\partial r^2} + \frac{1}{r} \frac{\partial f}{\partial r} + \frac{1}{r^2} \frac{\partial^2 f}{\partial \theta^2}$$

Simetría de los filtros

La simetría garantiza:

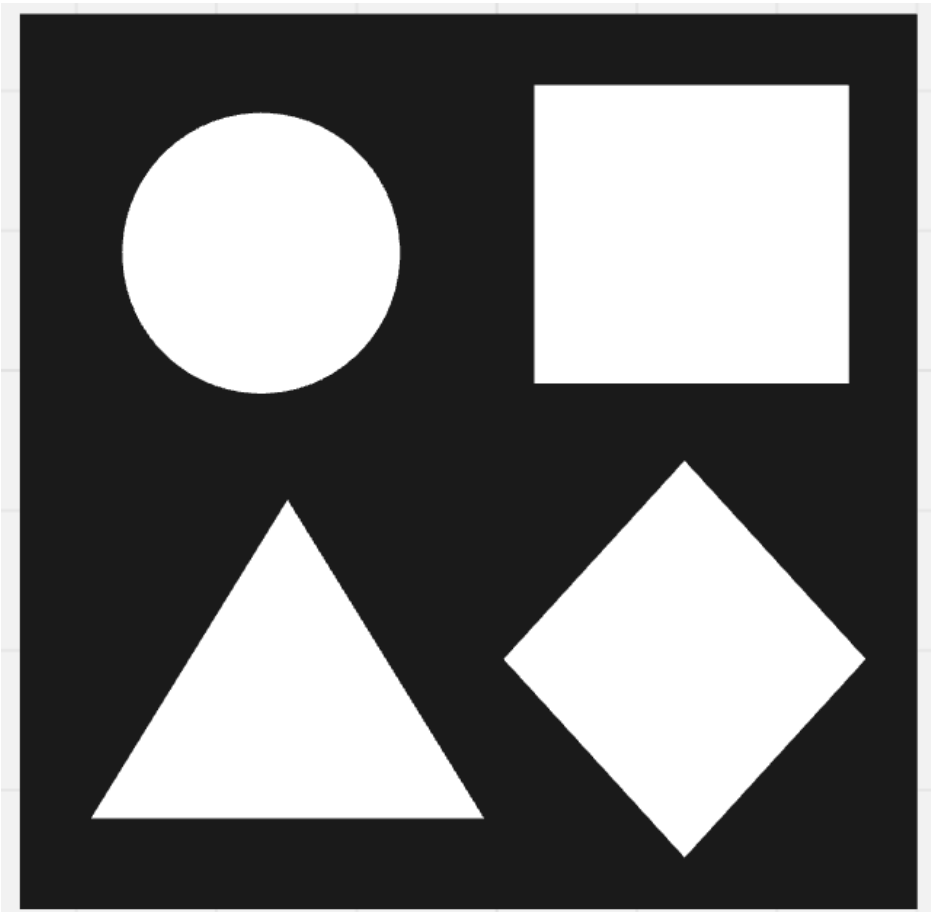
- 1.Respuesta equilibrada:** Ponderaciones negativas en un lado, positivas en el otro (como -1,1 o -2,2).
- 2.Sin sesgo direccional:** el filtro detecta los cambios de la misma manera si se mueve de izquierda a derecha o de derecha a izquierda.
- 3.Menos desplazamientos:** Un kernel simétrico no desplazará las características de la imagen resultante.



The image shows two handwritten 3x3 kernels in purple ink. The first kernel is labeled 'Sobel' and has values [-1, 0, 1] in the first row, [-2, 0, 2] in the second row, and [-1, 0, 1] in the third row. The second kernel is labeled 'Prewitt' and has values [-1, 0, 1] in the first row, [-1, 0, 1] in the second row, and [-1, 0, 1] in the third row.

$$\text{Sobel} \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$
$$\text{Prewitt} \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

Trabajemos con algunos kernels



Supongamos que queremos construir filtros para las siguientes operaciones.

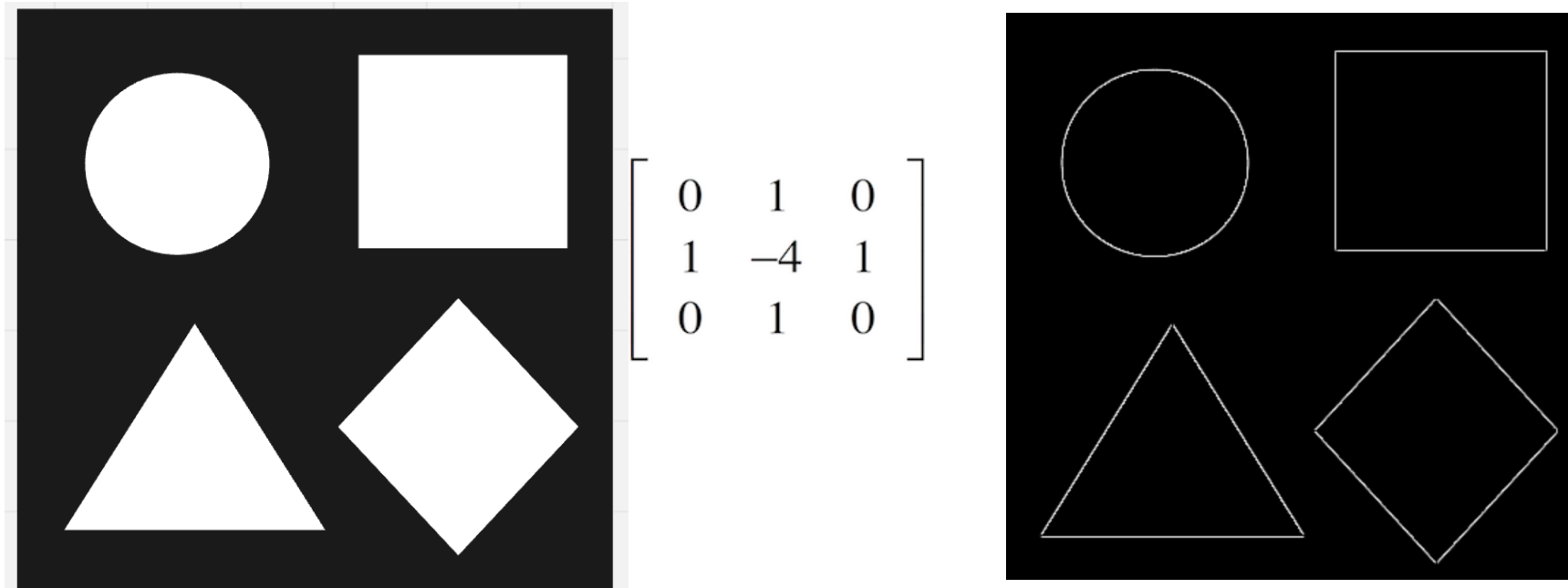
- 1.Transformación de identidad (sin cambios en la imagen)
- 2.Detección de bordes superiores
- 3.Detección de bordes inferiores
- 4.Detección de bordes derechos
- 5.Detección de bordes izquierdos
- 6.Detección de bordes inclinados
- 7.Detección de contorno
- 8.Desenfocar la imagen
- 9.Eliminación de ruido

Los siguientes filtros permiten las operaciones anteriores. Puede jugar con los valores de los filtros y ver qué sucede con su imagen de entrada aquí:

<https://setosa.io/ev/image-kernels/>

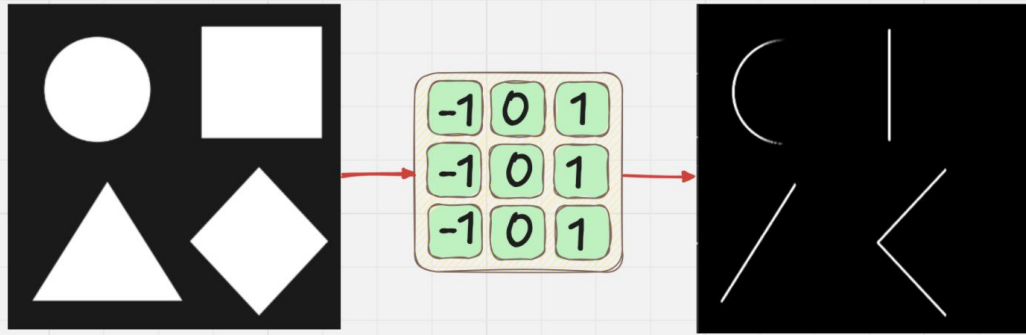
Taller Procesamiento de Imágenes

- Procesar múltiples filtros en una imagen entregada

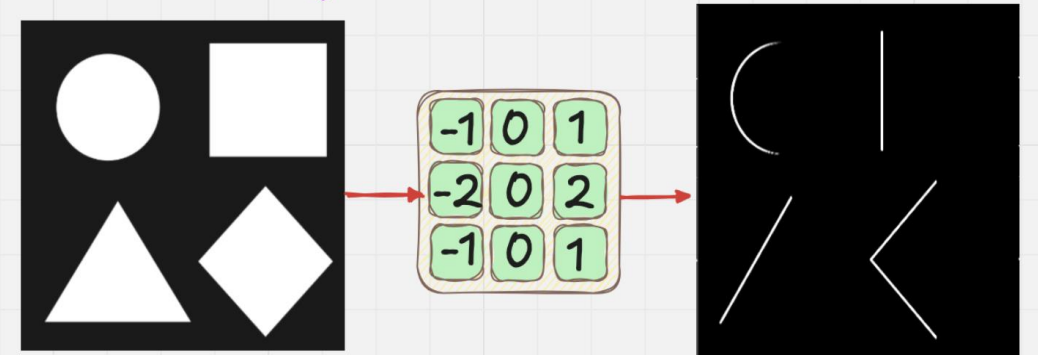


Testear Procesamiento de Imágenes

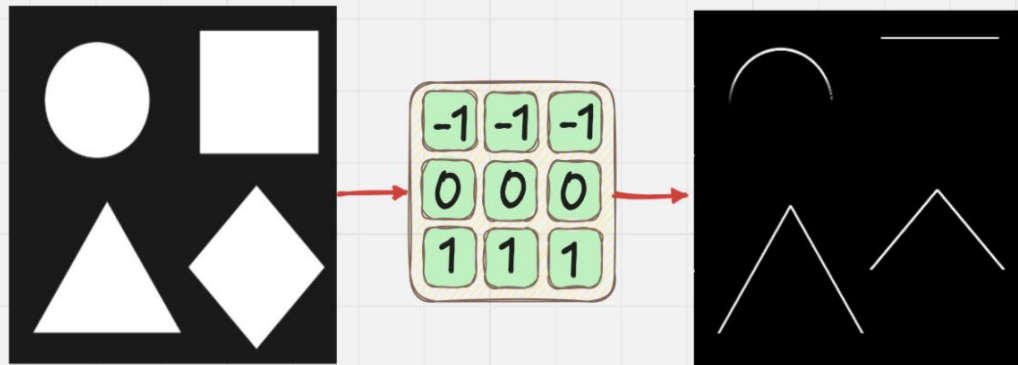
Left edge detection [Prewitt]



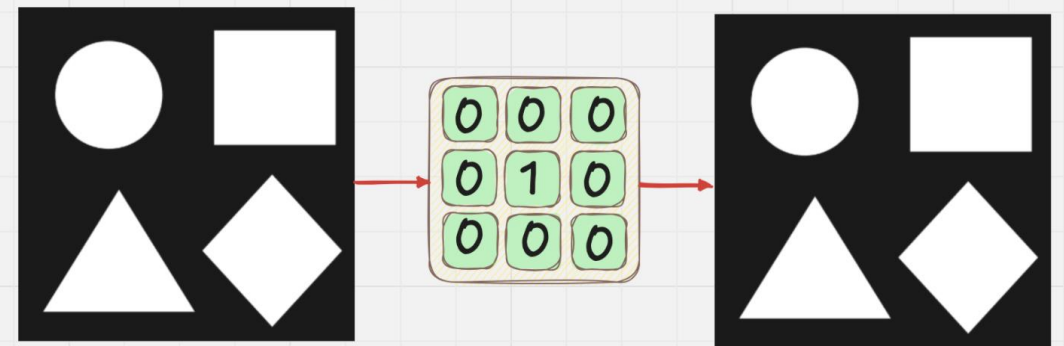
Left edge detection [Sobel]



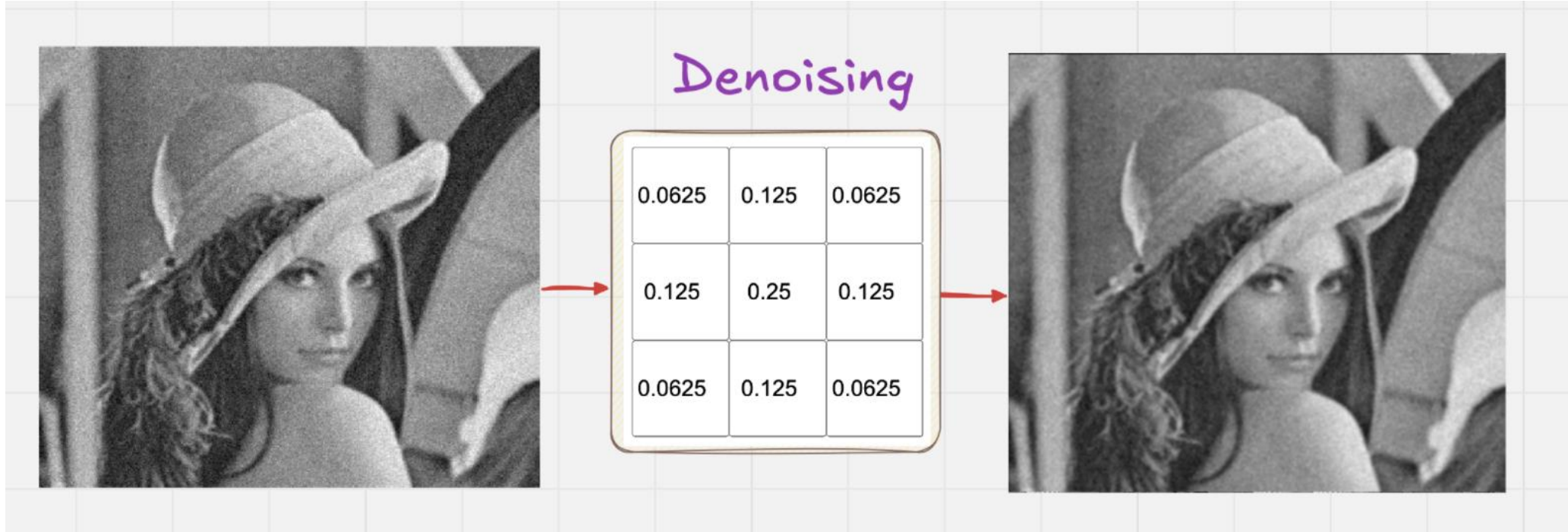
Top edge detection [Prewit]

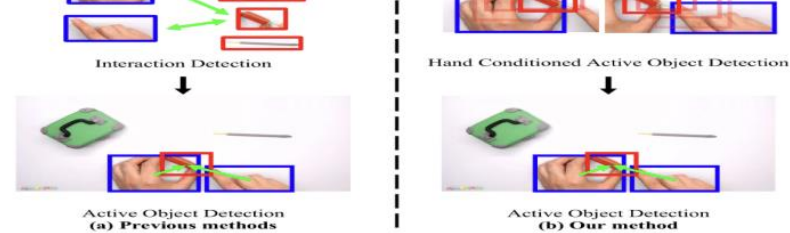


Identity transformation

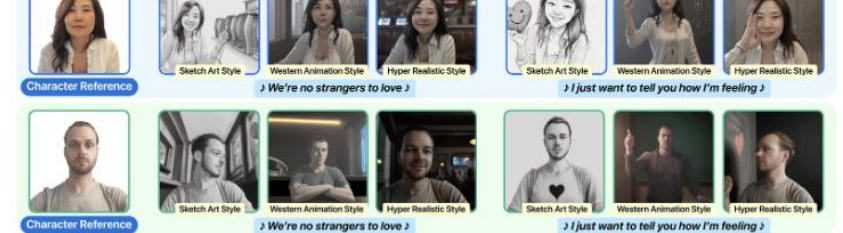


Eliminar el ruido





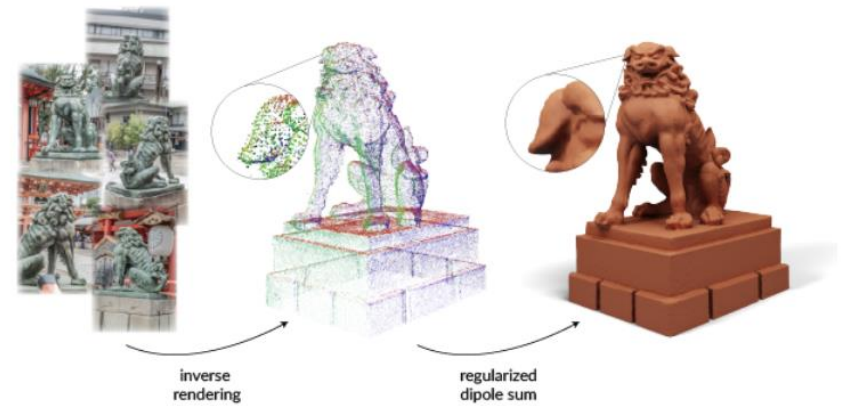
Sequential Voting with Relational Box Fields for Active Object Detection



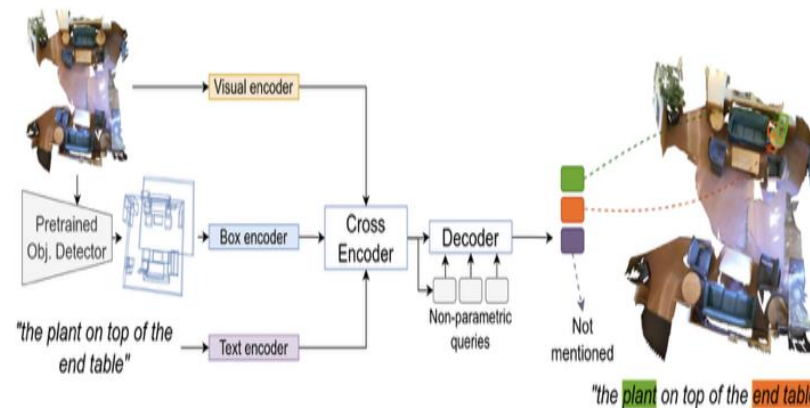
Secure & Personalized Music-to-Video Generation via CHARCHA



Depth-supervised NeRF: Fewer Views and Faster Training for Free



3D reconstruction with fast dipole sums



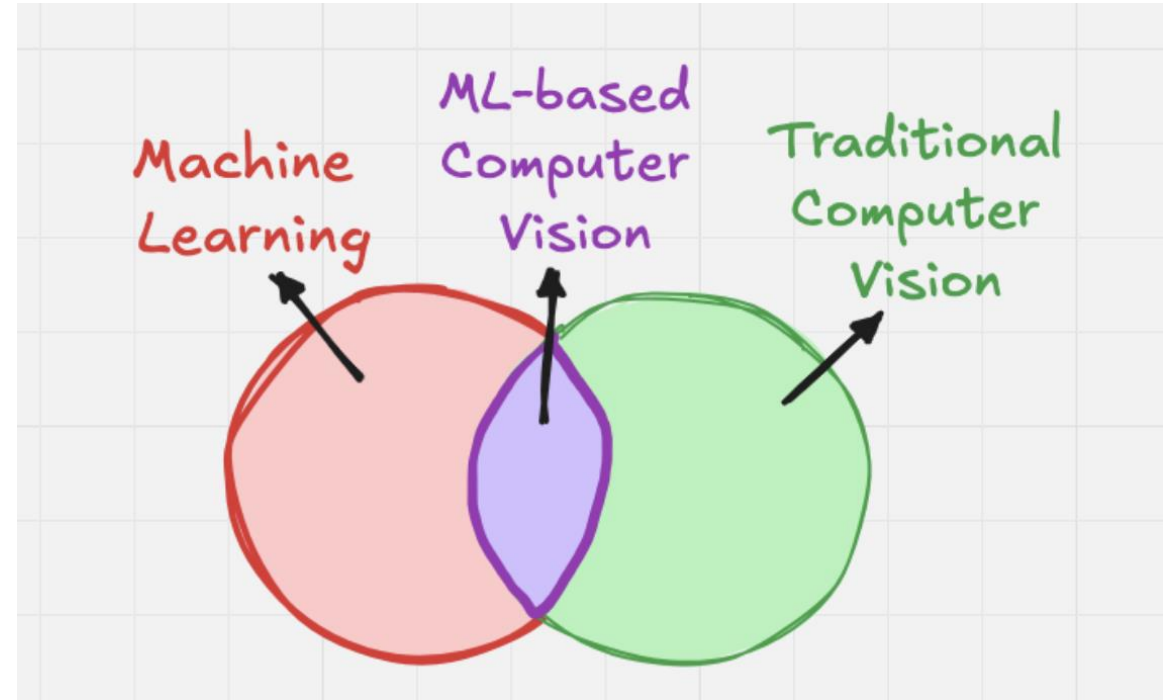
Bottom Up Top Down Detection Transformers for Language Grounding in Images and Point Clouds



Differentiable Raycasting for Self-supervised Occupancy Forecasting

Sistemas tradicionales basados en reglas vs basado en aprendizaje de máquinas (ML)

- En Machine Learning, dejamos que el modelo de ML descubra la lógica utilizando grandes cantidades de datos. No imponemos ninguna lógica estricta.



Porqué el enfoque de *Machine Learning* está explotando??

- Identifica a un gato (orejas, bigotes, ojos, etc.), entonces toda su vida estaría dedicada a crear miles de millones de condiciones que pueden suceder en imágenes de la vida real.
- ¿Qué pasa si hay un perro y un gato en una imagen?
- ¿Qué pasa si el gato está saltando una cerca?
- ¿Qué pasa si el gato está en un fondo de nieve?

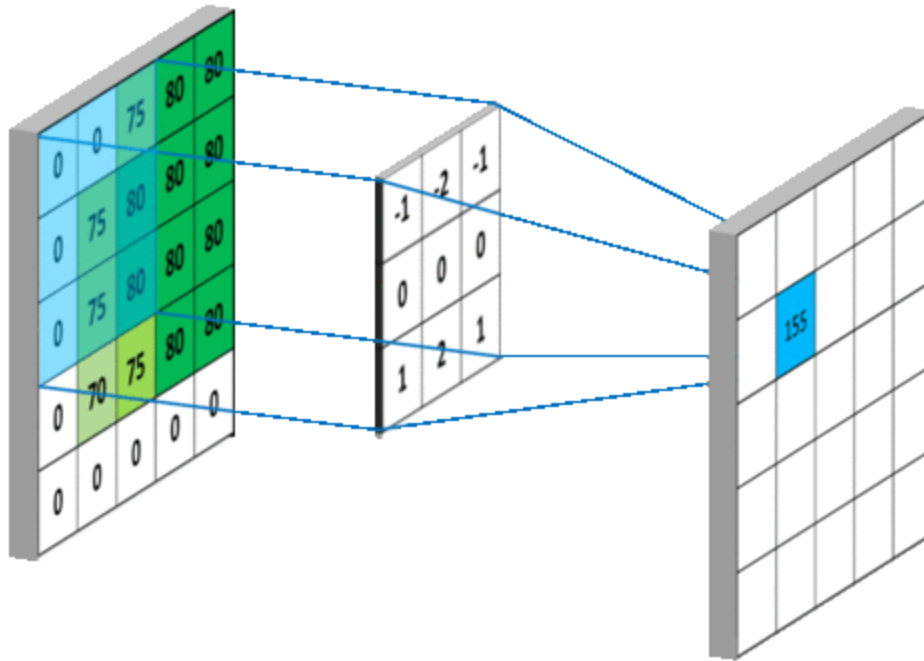


$$C(x, y) = \sum_{k_x=-K/2}^{K/2} \sum_{k_y=-K/2}^{K/2} f(k_x, k_y) I(x + k_x, y + k_y)$$

Introducción a convoluciones

- Una convolución 2D es una integral del producto de un filtro y una imagen de entrada calculada en varias posiciones a medida que el filtro se desliza a través de la entrada. La salida de la operación de convolución en la posición se puede escribir de la siguiente manera, donde el filtro es size :
- Este filtro convolucional a menudo se denomina kernel.

Convolucion



- La siguiente imagen es un ejemplo de operación de convolución utilizando un filtro de 3x3 aplicado en una matriz de 5x5.

CNN viene desde lo filtros

- Los filtros siguen siendo vitales para **la comprensión fundamental**: cómo funciona la detección de bordes, esquinas y texturas a un nivel bajo es exactamente lo que las CNN aprenden *automáticamente*.
- Además, los filtros clásicos siguen siendo útiles para **el preprocesamiento, los sistemas con restricciones computacionales** y para tareas **interpretables** o en tiempo real en las que las redes neuronales pesadas no son prácticas.

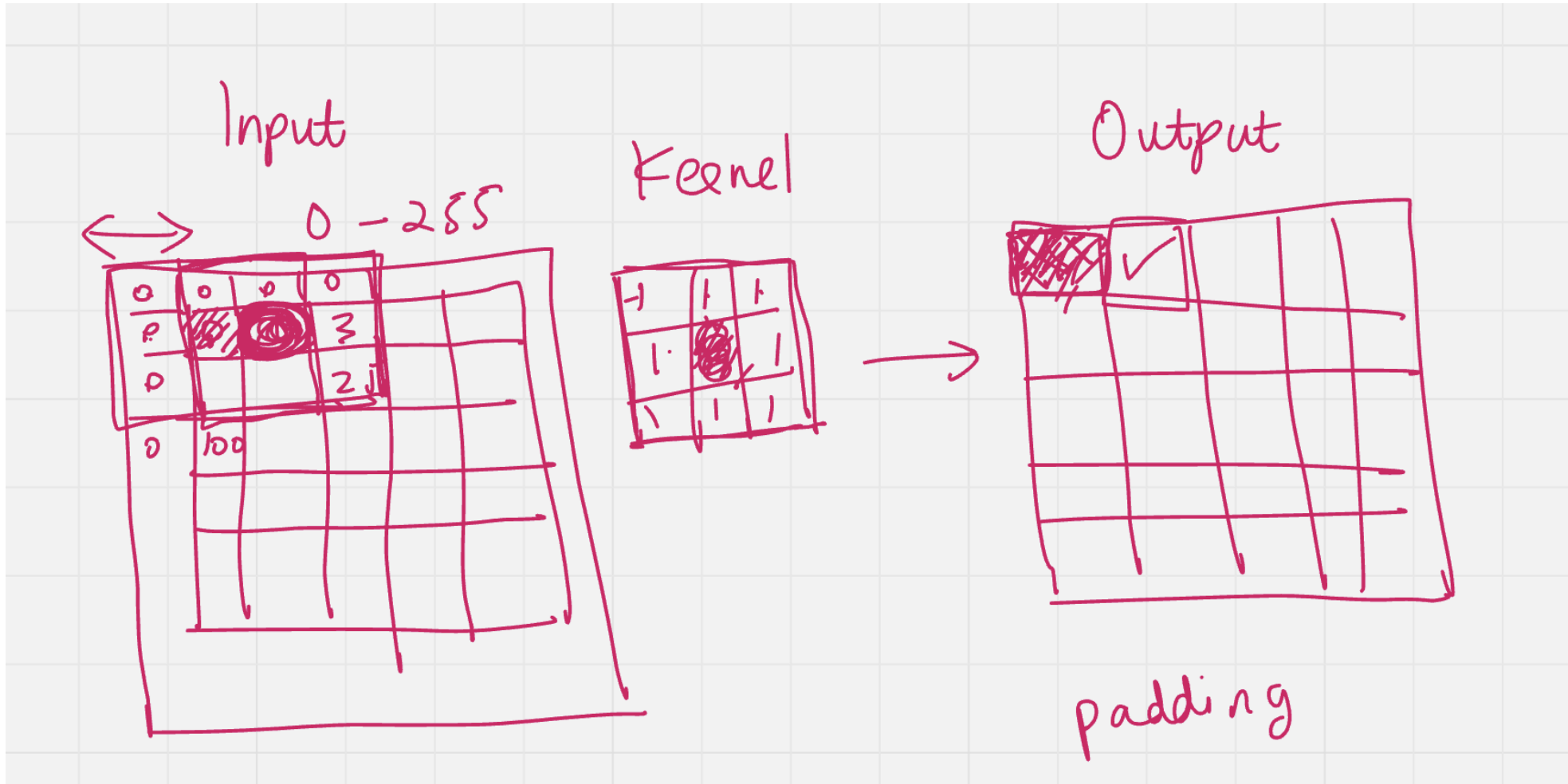
CNN

- **Las CNN usan filtros**, también conocidos como **kernels**. Sin embargo, a diferencia de los filtros tradicionales hechos a mano (por ejemplo, Sobel, Laplaciano).
- una CNN **aprende** sus filtros automáticamente a través de la retropropagación.
- Estos filtros aprendidos pueden volverse muy sofisticados, a menudo detectando bordes, esquinas, texturas o características más abstractas, sin que el usuario defina explícitamente ningún patrón en particular.

Filtro de imágenes - convolución

- En el sentido más simple, un filtro de imagen (*a menudo llamado kernel*) es una pequeña matriz que se coloca sobre una imagen (píxel por píxel) para producir alguna transformación.
- Para cada píxel, usted:
- **Superponga el filtro** en la vecindad local de ese píxel (normalmente una región 3×3 o 5×5).
- **Multiplique** cada entrada de filtro por la intensidad de píxeles correspondiente.
- **Sume** todos estos productos para producir una **sola intensidad nueva** (o valor de gradiente, o alguna otra medida).

Pizzara



Probar con visualizadores

- <https://ezyang.github.io/convolution-visualizer/index.html>

Kernel size:

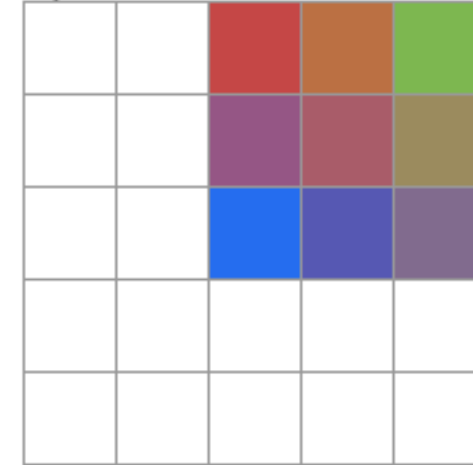
Kernel size:

adding:

Dilation:

Stride:

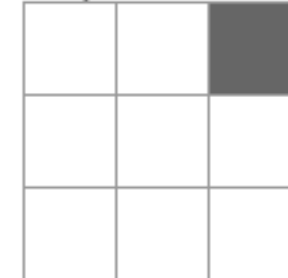
Input (5 × 5):



Weight (3 × 3):

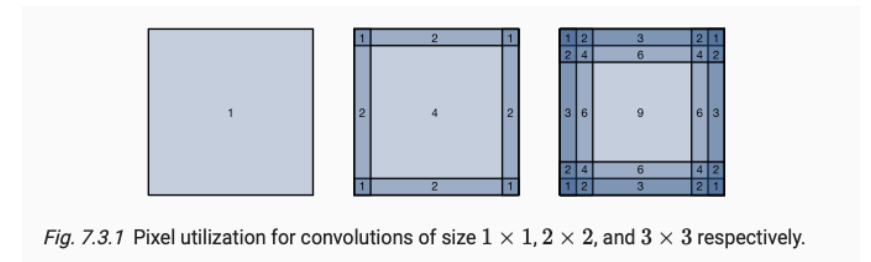
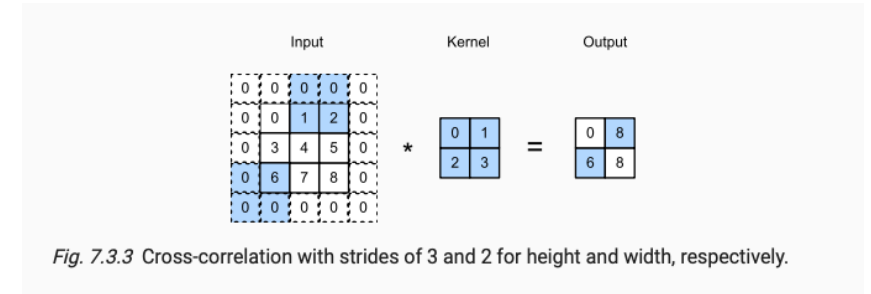


Output (3 × 3):



Padding, Stride, y otras técnicas para la Convolución

- Este proceso de **convolución** es una operación matemática en la que una pequeña matriz (kernel) se desliza sobre una imagen, realizando un producto de puntos con la región superpuesta para generar una nueva matriz que representa características transformadas.



Hiperparámetros

Input Size: 5

Padding: 0

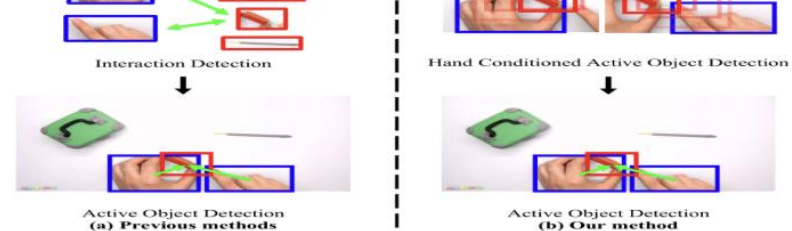
Kernel Size: 2

Stride: 1

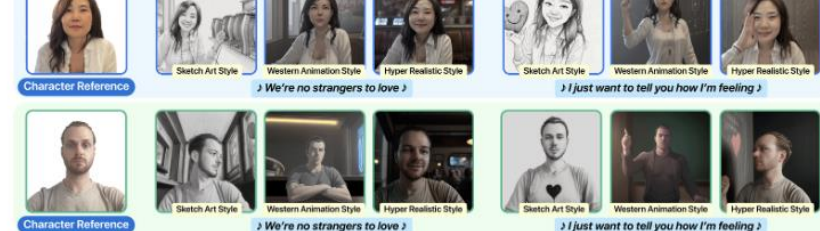
Input (5, 5)
After-padding (5, 5)

Output (4, 4)

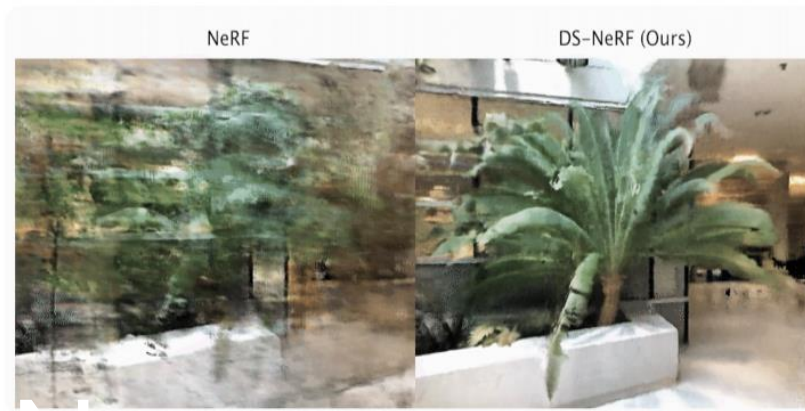
 *Hover over the matrices to change kernel position.*



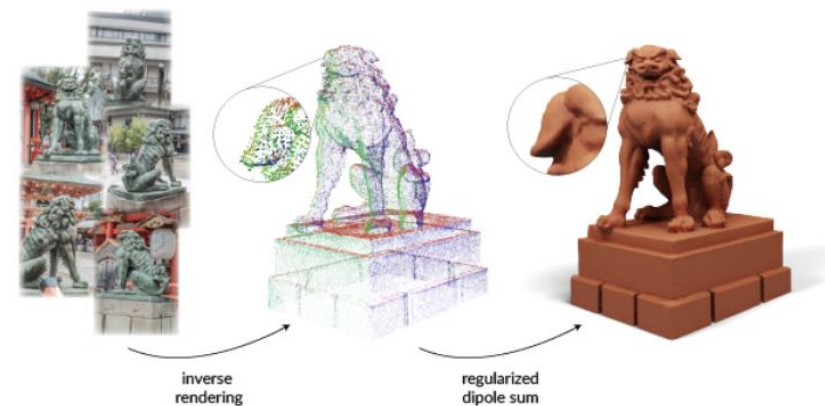
Sequential Voting with Relational Box Fields for Active Object Detection



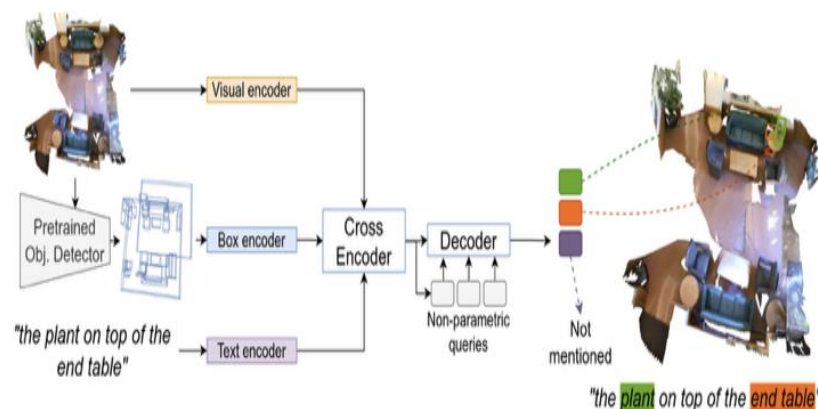
Secure & Personalized Music-to-Video Generation via CHARCHA



Depth-supervised NeRF: Fewer Views and Faster Training for Free



3D reconstruction with fast dipole sums



Bottom Up Top Down Detection Transformers for Language Grounding in Images and Point Clouds

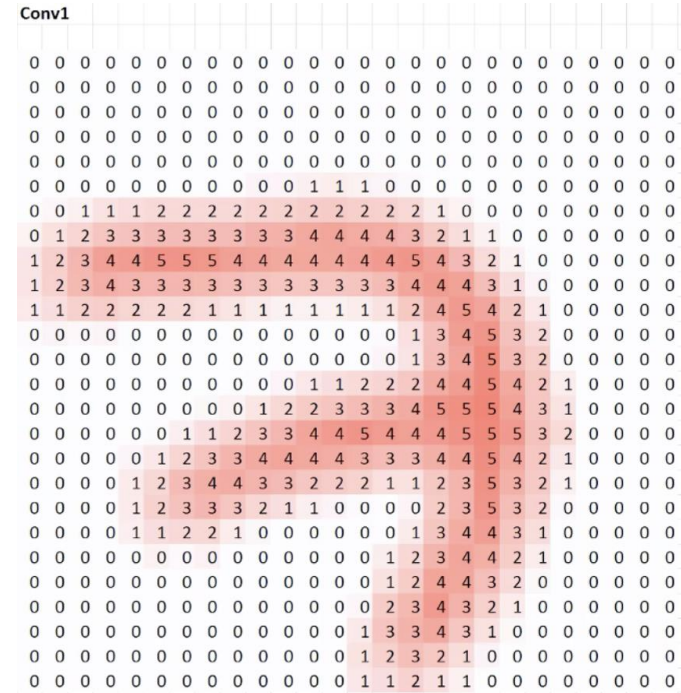
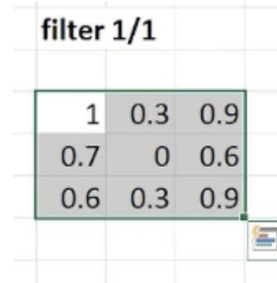
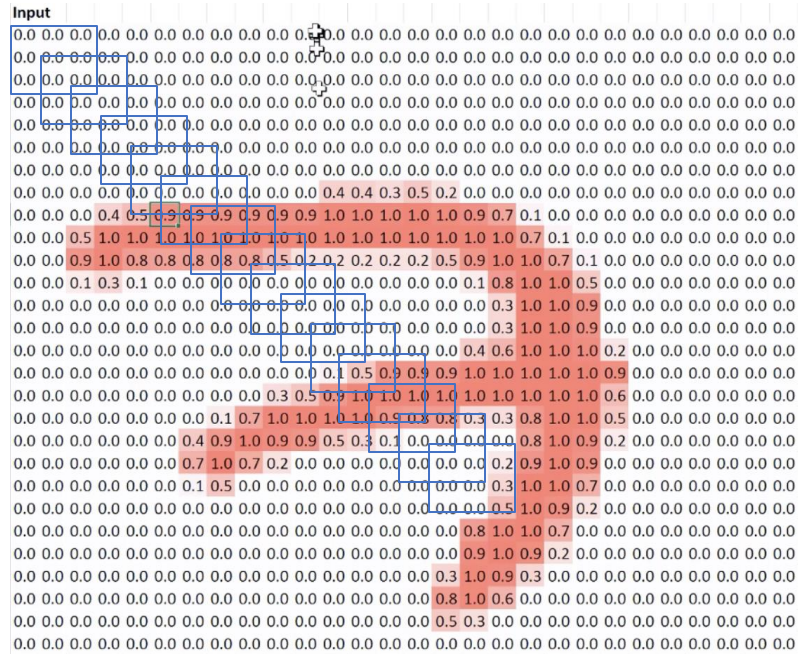


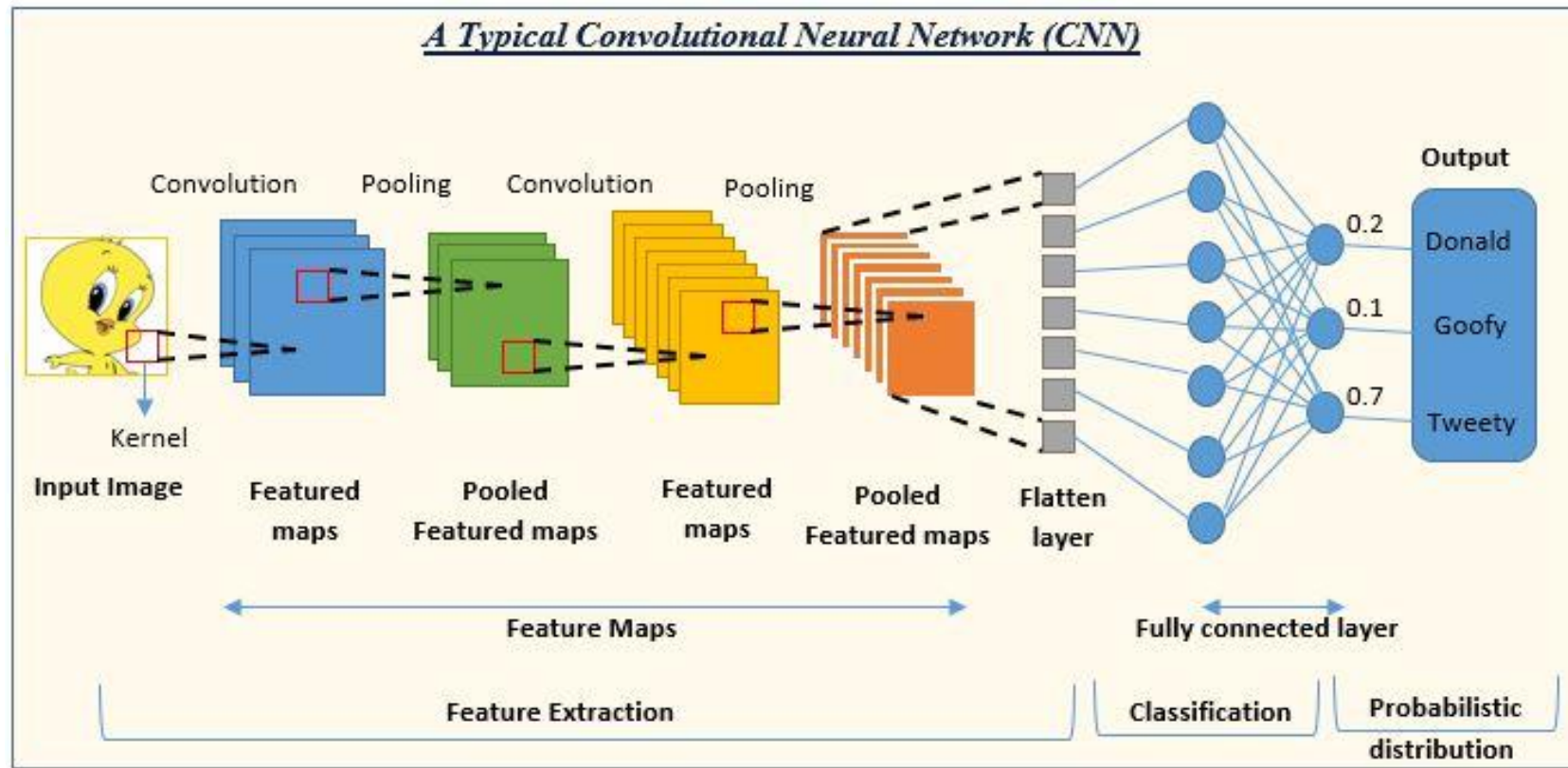
Differentiable Raycasting for Self-supervised Occupancy Forecasting

CNN

Una red neuronal convolucional (CNN) es un tipo de red neuronal feedforward que aprende características a través de la optimización del filtro (o kernel).

Ejemplo con el mismo 7





CNN

Introducción a convoluciones 2D

